ТЕХНИЧЕСКИЙ ГАЙД

LitGraphAgent

Автономный агент для сбора, фильтрации и построения графов литературы в Obsidian

Полное введение в проект для новых участников, преподавателей и инженеров.
Архитектура, пять агентов, логика пайплайна, метрики, запуск и глоссарий.

Команда AI-RIldium | ноябрь 2026
github.com/Icold21/LitGraphAgent | MIT License

Документ объясняет не только «что сделано», но и «почему это работает»: от precision и recall до того, как LLM выбирает поисковый движок.

Содержание

1. Зачем этот документ

1.1. Что такое LitGraphAgent в одном абзаце
1.2. Какую проблему решает проект
1.3. Для кого проект
1.4. Что система делает и чего не делает
1.5. Как читать документ
1.6. Краткая карта репозитория
1.7. Главная мысль

2. Базовые понятия без предположений

2.1. LLM и всё, что вокруг
2.2. Граф знаний
2.3. Obsidian и Markdown
2.4. Агент и оркестрация
2.5. Поиск, отбор и ранжирование
2.6. Качество, тесты и CI
2.7. Что важно понять прямо сейчас

3. Архитектура проекта

3.1. Общая картина
3.2. Почему именно пять ролей
3.3. LangGraph: диспетчер, а не «мозг»
3.4. ResearchState: что передаётся между узлами
3.5. MemorySaver и _state.json
3.6. Инварианты архитектуры
3.7. Что происходит при ошибке

4. Подробный разбор агентов

4.1. LeadAIAgent (LiteratureAgent)
4.2. RetrievalAgent
4.3. PromptAgent и LLM-слой
4.4. GraphAgent
4.5. EvaluationAgent

5. Сквозные сценарии и данные

5.1. Сценарий create: из темы в vault
5.2. Сценарий update: продолжить базу
5.3. Сценарий deep: PDF и grounding
5.4. Сценарий check, list, delete
5.5. Модель данных
5.6. _state.json
5.7. Структура vault
5.8. Почему vault, а не база данных
5.9. Что происходит при повторном запуске

6. Как устроены поиск, LLM и граф

6.1. Поиск и выбор движков
6.2. LLM-слой
6.3. Граф и каноникализация

7. QA, тесты и бенчмарк

7.1. Зачем отдельный QA-агент
7.2. Проверки vault
7.3. Проверки состояния
7.4. Тесты и CI
7.5. SciERC: что это
7.6. Что именно измеряет наш harness
7.7. Exact, relaxed и content-only
7.8. Результаты на 20 абстрактах
7.9. Что дал prompt v2

7.10. Опорные точки
7.11. Ограничения бенчмарка
7.12. Как читать эту таблицу

8. Метрики с нуля

8.1. Задача извлечения концептов
8.2. TP, FP, FN и TN
8.3. Precision
8.4. Recall
8.5. F1
8.6. Почему не accuracy
8.7. Exact и relaxed
8.8. Micro и macro
8.9. Content-only
8.10. MRR
8.11. Hits@k
8.12. nDCG
8.13. MAP
8.14. Triple F1
8.15. Coverage, orphan rate, unresolved link rate
8.16. Как читать таблицу LitGraphAgent
8.17. Частые ошибки
8.18. Главное

9. Запуск, код и команда

9.1. Установка
9.2. Настройка .env
9.3. Проверка окружения
9.4. Основные команды
9.5. Тесты и линтер
9.6. Бенчмарк
9.7. Типичные проблемы
9.8. Карта кода
9.9. Как добавить нового агента
9.10. Как добавить поисковый движок
9.11. Как изменить промпт
9.12. Команда и роли
9.13. Дорожная карта
9.14. Риски
9.15. Что считать успехом

10. Глоссарий, FAQ и приложение

10.1. Глоссарий
10.2. FAQ
10.3. Приложение A. Пример _state.json
10.4. Приложение B. Пример заметки источника
10.5. Приложение C. Пример концепт-хаба
10.6. Приложение D. Пример QA-отчёта
10.7. Приложение E. Промпт v2
10.8. Приложение F. Шпаргалка команд
10.9. Что читать дальше
10.10. Заключение

1. Зачем этот документ

Этот документ — технический гайд по LitGraphAgent для человека, который никогда не участвовал в проекте и хочет за один-два дня понять, что здесь происходит, почему это работает и где что лежит. Он не предполагает глубокой подготовки по машинному обучению: все ключевые понятия объясняются с нуля, на бытовых примерах, а затем постепенно привязываются к коду.

Документ устроен так, чтобы его можно было читать последовательно, но при этом он остаётся справочником. Если вы менеджер или преподаватель, начните с разделов 1–3 и 7–8. Если вы инженер, которому нужно быстро включиться в код, читайте разделы 3–6, затем 9. Если вы новичок в ML, первые три раздела и раздел 8 дадут необходимый минимум.

1.1. Что такое LitGraphAgent в одном абзаце

LitGraphAgent — это автономный исследовательский агент, который получает тему и требования к литературе, сам ищет научные статьи в нескольких источниках, отбирает релевантные работы, извлекает из них ключевые концепты, строит граф связей «статья → концепт» и «статья → статья», а затем записывает результат в локальную базу Obsidian: атомарные заметки, концепт-хабы, библиографию, обзорный синтез и журнал работы агентов.

Система не является «поисковиком» и не заменяет эксперта. Она автоматизирует рутину: сбор кандидатов, первичную фильтрацию, ранжирование, структурирование и оформление. Человек всё равно должен проверить источники, особенно если результат идёт в академический текст.

1.2. Какую проблему решает проект

У исследователя есть три повторяющиеся проблемы.

1. **Информационный шум.** По многим темам выходят сотни и тысячи работ. Просмотреть их вручную невозможно, а поиск по ключевым словам даёт много нерелевантного.
2. **Потеря связей.** Даже если статьи найдены, их нужно связать между собой: какая работа использует какой метод, какой концепт встречается в нескольких статьях, кто на кого ссылается. В обычной папке с PDF эти связи теряются.
3. **Рутина оформления.** Библиография, единый формат цитирования, ссылки, конспекты и оглавление отнимают часы, хотя почти не требуют творчества.

LitGraphAgent пытается закрыть все три пункта одним проходом: тема → проверенные источники → граф концептов → готовая библиография и обзор.

1.3. Для кого проект

- **Студенты и аспиранты** — быстро собрать литературу по новой теме.
- **Исследователи** — держать локальную базу знаний, которая не зависит от облачного сервиса.
- **R&D-команды** — сохранять общий граф источников и концептов по проекту.
- **Преподаватели** — показывать, как устроен пайплайн «поиск → отбор → извлечение знаний → граф».

1.4. Что система делает и чего не делает

Делает:

- ищет статьи в Semantic Scholar, arXiv и открытом вебе;
- выбирает поисковые движки с помощью LLM и валидирует это решение;
- фильтрует кандидатов по теме и требованиям;
- ранжирует статьи по наукометрическим сигналам и отсекает хвост;
- извлекает из абстракта (а в deep-режиме — из PDF)summary, findings, methodology, limitations и концепты;
- канонизирует концепты, чтобы `GNN`, `GNNs` и `Graph Neural Networks` не превращались в три разных узла;
- строит заметки Obsidian, концепт-хабы, `[[wikilinks]]`, внешние заглушки и библиографию;
- сохраняет состояние в `_state.json` и поддерживает инкрементальные обновления;
- проверяет целостность vault и состояния через QA-агент;
- измеряет качество извлечения концептов на SciERC.

Не делает:

- не гарантирует, что источник качественный или peer-reviewed;
- не заменяет чтение статьи экспертом;
- не извлекает типизированные отношения уровня SciERC (это ограничение честно зафиксировано в бенчмарке);
- не выполняет end-to-end оценку качества поиска (precision@k, nDCG) — это задача следующего этапа;
- не является облачным сервисом: всё работает локально, с локальной или внешней LLM по выбору пользователя.

1.5. Как читать документ

- **Раздел 2** — минимальный словарь: LLM, промпт, граф знаний, Obsidian, агент, оркестрация. Если вы уже знаете эти слова, можно пропустить.
- **Раздел 3** — общая архитектура и схема взаимодействия пяти агентов.
- **Раздел 4** — подробный разбор каждого агента: что он получает, что отдаёт, какие методы вызываются и где код.
- **Раздел 5** — сквозные сценарии и данные: create, update, deep, check, `_state.json`, структура vault.
- **Раздел 6** — три «начинки»: поиск и ранжирование, LLM-слой, граф и каноникализация.
- **Раздел 7** — QA, тесты, CI и бенчмарк SciERC с результатами.
- **Раздел 8** — метрики с нуля: precision, recall, F1, exact/relaxed, MRR, Hits@k, nDCG, triple F1. Это то, что нужно, чтобы читать таблицы и не путать разные задачи.

- **Раздел 9** — как запустить, где что лежит, команда, роли, дорожная карта.
- **Раздел 10** — глоссарий, FAQ и приложение с примерами файлов.

1.6. Краткая карта репозитория

```
lit_graph/
  agent/        роли, оркестрация, состояние, агенты
  search/       клиенты arXiv/S2/web, каталог движков, роутер, ранжирование
  llm/          провайдеры LLM и все промпты
  storage/      запись Obsidian vault, состояние, имена файлов
  qa/           проверки vault и состояния
  cli.py        командная строка
  config.py     настройки из .env
  models.py     модели данных Paper, ConceptHub, GraphEdge, LiteratureBaseState
tests/          55 тестов
benchmarks/     SciERC-бенчмарк и сырые результаты
docs/           архитектура, отчёты, презентации, этот гайд
```

1.7. Главная мысль

LitGraphAgent — это не «магический ИИ, который всё знает». Это конвейер, в котором каждый шаг можно проверить: какой движок выбран, какие статьи отброшены, какие концепты извлечены, какие связи построены, что нашёл QA. Именно прозрачность шагов делает систему пригодной для учебного и исследовательского контекста.

2. Базовые понятия без предположений

Этот раздел объясняет слова, которые дальше встречаются постоянно. Если какой-то термин кажется очевидным, его можно пропустить, но лучше вернуться к нему, когда он появится в коде.

2.1. LLM и всё, что вокруг

LLM (large language model) — большая языковая модель. Её можно представить как функцию, которая получает текст и предсказывает продолжение. Внутри — много параметров, обученных на больших корпусах текста. Модель не «знает» факты как база данных: она выучила статистические закономерности и может ошибаться, особенно в узких областях.

В проекте LLM используется не для «чата», а как исполнитель конкретных операций:

- прочитать описание поисковых движков и выбрать подходящие;
- проверить, соответствует ли статья теме и требованиям;
- извлечь из абстракта summary, findings, methodology, limitations и концепты;
- написать определения концептов;
- собрать библиографию и обзор.

Промпт (prompt) — текст, который мы отправляем модели. Промпт — это не «вопрос» в бытовом смысле, а инструкция: роль, входные данные, правила и формат ответа. В LitGraphAgent почти все промпты требуют строгий JSON, чтобы результат можно было разобрать программно, а не читать глазами.

Токен — минимальная единица текста для модели: слово, часть слова, знак. Модель генерирует токены по одному. Ограничение на количество токенов означает, что очень длинный текст может не поместиться в контекст, а очень длинный ответ может обрезаться.

Контекст — весь текст, который модель видит в одном запросе: системная инструкция, промпт, фрагмент статьи. Если в контекст положить слишком много, модель может потерять важное или упереться в лимит.

Температура — параметр случайности. Чем выше, тем более разнообразные ответы; чем ниже, тем более предсказуемые. В проекте используется низкая температура (0.2), чтобы ответы были стабильнее.

JSON — текстовый формат вида `{"key": "value"}`. Он удобен тем, что его можно строго разобрать в программе. Модель может добавить markdown-обёртку,

лишний текст или незакрытую скобку, поэтому в проекте есть отдельный «санитайзер» JSON.

Ollama — локальный сервер, который запускает модели на вашем компьютере и предоставляет OpenAI-совместимый API. В проекте через Ollama запускаются `qwen2.5:3b` и `qwen3.5:4b`. Локальная модель не отправляет текст в облако, но требует GPU/CPU и времени.

Провайдер LLM — конкретный сервис или локальный сервер, к которому обращается проект: DeepSeek, OpenAI, Groq, OpenRouter, Ollama, vLLM или любой OpenAI-совместимый endpoint. Провайдер скрыт за общим интерфейсом, поэтому замена модели — это изменение `.env`, а не переписывание агентов.

Эмбеддинги — числовые векторы, которые представляют смысл текста. В этом проекте они не используются для поиска: поиск идёт через API научных баз, а связи строятся по концептам и цитированиям. Это важно, чтобы не искать «векторную магию» там, где её нет.

2.2. Граф знаний

Граф — структура из узлов и рёбер. Узел — объект, ребро — связь.

Граф знаний (knowledge graph, KG) — граф, где узлы обозначают сущности (статьи, концепты, авторы, методы), а рёбра — отношения между ними. Часто используют формат троек:

```
(голова, отношение, хвост)
(Induction Heads, part_of, Transformer Circuits)
(Paper A, cites, Paper B)
```

Узел (node) — вершина графа. В LitGraphAgent узлы — это источники (`Sources/`) и концепты (`Concepts/`).

Ребро (edge) — связь. В проекте есть:

- `paper → concept` — статья упоминает концепт;
- `paper → paper` — цитирование, тип `cites`;
- `concept → paper` — обратная ссылка, которая генерируется в концепт-хабе.

Триплет (triple) — тройка «голова, отношение, хвост». В SciERC и других KG-бенчмарках качество часто измеряют именно на тройках.

Онтология — правила, какие типы узлов и рёбер вообще допустимы. В проекте онтология задана структурой vault: `literature_note`, `concept_hub`, `external_reference`, `synthesis_hub`, `master_bibliography`, `agent_activity`.

Каноникализация — приведение разных написаний одной сущности к одному каноническому виду. Пример: `GNN`, `GNNs`, `Graph Neural Networks` должны стать одним концептом. Без каноникализации граф распадается на дубли.

Покрытие (coverage) — доля gold-сущностей или связей, которые попали в граф. Если в статье 10 концептов, а система извлекла 5, coverage = 0.5.

Сирота (orphan) — узел без входящих и исходящих ссылок. В Obsidian это заметка, до которой нельзя дойти по ссылкам.

Unresolved link — `[[ссылка]]`, которая не находит заметку. В Obsidian такая ссылка выглядит как «битая».

2.3. Obsidian и Markdown

Markdown — простой текстовый формат разметки. Заголовки `#`, списки `-`, выделение `**жирным**`, ссылки `[текст](url)`.

Obsidian — приложение для ведения заметок в виде локальной папки Markdown-файлов. Оно показывает граф связей и поддерживает `[[wikilinks]]`.

Vault — папка с заметками Obsidian. В LitGraphAgent vault — это результат работы: `Sources/`, `Concepts/`, библиография, обзор, состояние.

Frontmatter — YAML-блок в начале заметки между `---`. В нём хранятся метаданные: `title`, `type`, `year`, `citation_count`, `node_weight` и т.д.

Wikilink — ссылка вида `[[Название заметки]]`. Она связывает заметки внутри vault и отображается в graph view.

Graph view — режим Obsidian, где заметки показаны узлами, а ссылки — рёбрами. Именно его пользователь видит как «карту литературы».

Stub (заглушка) — лёгкая заметка для внешней статьи, которая упоминается в цитированиях, но не была загружена. Нужна, чтобы `[[wikilink]]` не повисал.

2.4. Агент и оркестрация

Агент — программный компонент, который выполняет ограниченную роль: получает данные, принимает решение, возвращает результат. В LitGraphAgent пять ролей: Lead, Retrieval, Prompt, Graph, QA.

Оркестрация — управление тем, в каком порядке агенты вызываются и как передаются данные. В проекте оркестратор — LangGraph.

LangGraph — библиотека для описания workflow как графа: узлы — шаги, рёбра — переходы, условные рёбра — ветвления. Важно: LangGraph не содержит

бизнес-логику. Он только маршрутизирует вызовы агентов.

Состояние (state) — данные, которые передаются между узлами. В проекте это `ResearchState`: тема, требования, кандидаты, выбранные статьи, текущая статья, страницы PDF, activity, QA-отчёт.

Reducer — функция, которая объединяет старое и новое значение поля. Для `activity` используется append-reducer: каждый узел добавляет событие, а не перезаписывает журнал.

Checkpoint — сохранённая точка состояния. `MemorySaver` хранит состояние конкретного запуска в памяти; `_state.json` — durable-хранилище между запусками.

2.5. Поиск, отбор и ранжирование

Retrieval (поиск) — получение кандидатов из внешних источников.

Semantic Scholar — академическая поисковая база с метаданными и цитированиями. Даёт citation count и influential citation count.

arXiv — preprint-сервер. Хорош для свежих работ, но не гарантирует peer-review.

Open web — веб-поиск и парсинг страниц. Нужен, когда академические API не дают результат, но качество шумнее.

Дедупликация — удаление одинаковых статей, пришедших из разных движков. Сравниваются DOI, URL и нормализованное название.

Ранжирование — сортировка кандидатов по полезности. В проекте используется наукометрический скор: citation velocity, influential ratio, log-scaled impact.

Elbow cutoff — эвристика «локтя»: отсортированные статьи делятся на сильную голову и слабый хвост; хвост отсекается. Это не «магия», а способ не тащить в базу случайные low-impact работы.

Fallback — запасной вариант. Если primary-движок дал мало результатов, система пробует fallback-движок.

Sub-query — уточняющий подзапрос, который генерирует LLM, если после первого раунда статей недостаточно.

2.6. Качество, тесты и CI

QA (quality assurance) — проверка результата: ссылки, frontmatter, дубли,

сироты, целостность связей.

Тест — автоматическая проверка поведения кода. В проекте 55 тестов.

pytest — фреймворк для тестов на Python.

ruff — быстрый линтер и форматтер для Python. В CI он падает, если есть ошибки стиля.

CI (continuous integration) — автоматический запуск тестов и линтера на каждый push/pull request. В проекте GitHub Actions проверяет Python 3.10–3.13.

Бенчмарк — стандартизированный набор данных и протокол оценки. В проекте используется SciERC.

Precision, recall, F1 — метрики качества. Подробно разбираются в разделе 8; здесь достаточно интуиции: precision — «сколько из найденного правильное», recall — «сколько из правильного найдено», F1 — их баланс.

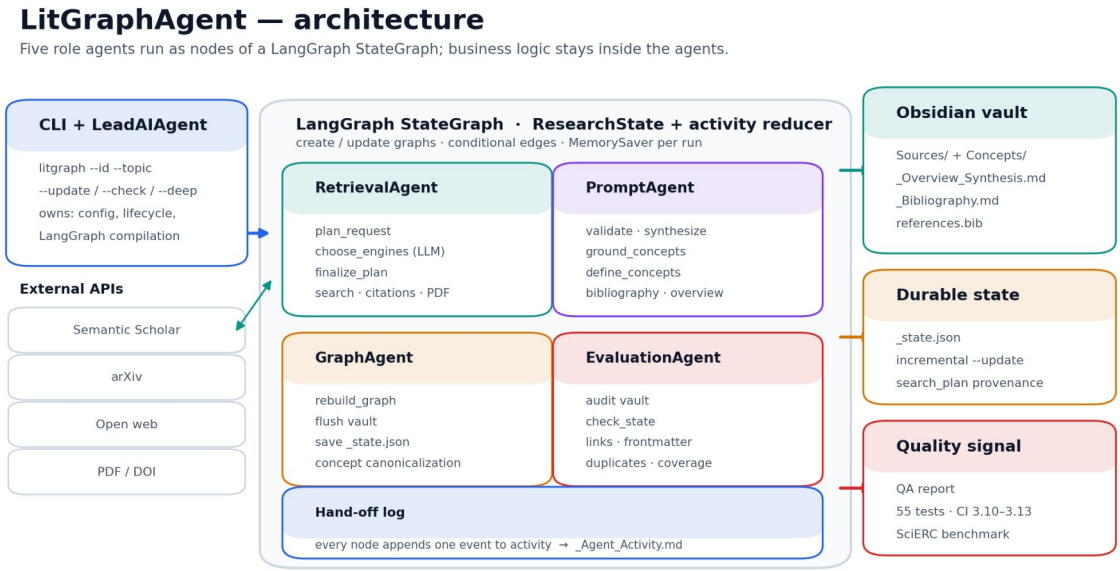
2.7. Что важно понять прямо сейчас

1. LLM — вероятностный компонент, а не база фактов. Поэтому в проекте есть валидация, JSON-санитайзер, fallback-и и QA.
2. Граф знаний — это узлы и рёбра, а не просто папка с файлами. Поэтому важны каноникализация, ссылки и дедупликация.
3. LangGraph — это диспетчер, а не «мозг». Бизнес-логика живёт в агентах.
4. `_state.json` — источник правды между запусками; без него `--update` не смог бы продолжать базу.
5. Бенчмарк измеряет только то, что система реально предсказывает. Если пайплайн не предсказывает типы отношений, relation F1 не считается.

3. Архитектура проекта

3.1. Общая картина

LitGraphAgent построен как команда из пяти ролей, которые работают в одном пайплайне. Пользователь запускает CLI, Lead-агент собирает LangGraph-workflow и передаёт работу специалистам. Каждый шаг — отдельный узел графа; каждый переход — ребро; каждый вызов агента оставляет событие в журнале `activity`.



Архитектура LitGraphAgent

Пять ролей:

Роль	Агент	Что делает
Lead AI Engineer	`LeadAIAgent` (`LiteratureAgent`)	Жизненный цикл базы, сборка workflow, CLI, состояние
Data / IR Engineer	`RetrievalAgent`	Поиск, дедупликация, цитирования, PDF, кэш
LLM Engineer	`PromptAgent`	Валидация, синтез, концепты, библиография, обзор
Graph & Knowledge Engineer	`GraphAgent`	Онтология, каноникализация, запись vault, ссылки
QA / Evaluation Engineer	`EvaluationAgent`	Аудит vault и состояния, отчёт QA

Важно: это не пять независимых ботов, которые «переписываются». Это пять наборов методов, которые вызываются из одного workflow. Роли разделены по ответственности, а не по процессам.

3.2. Почему именно пять ролей

Разделение возникло из реальных зон:

- **Поиск** требует знания API, ретраев, дедупликации и ранжирования. Это отдельная инженерная зона.
- **LLM-операции** требуют промптов, батчинга, JSON-устойчивости и валидации. Это другая зона.
- **Граф** требует онтологии, каноникализации, имён файлов и Obsidian.
- **QA** должен быть независимым: он не пишет данные, а проверяет то, что написали другие.
- **Lead** координирует, но не подменяет специалистов.

Такое разделение позволяет менять один слой, не переписывая остальные.

Например, заменить arXiv-клиент, не трогая промпты; или добавить новую

проверку QA, не меняя граф.

3.3. LangGraph: диспетчер, а не «мозг»

`LiteratureWorkflow` (`lit_graph/agent/workflow.py`) собирает

`StateGraph(ResearchState)`. Каждый узел — это обёртка `_step(recipient,

action, fn)`, которая:

1. вызывает метод агента;
2. замеряет время;
3. добавляет событие в `activity`;
4. возвращает только изменённые ключи состояния.

Бизнес-логика остаётся в агентах. Узел не решает, как искать; он вызывает

`self.retrieval.search(...)`. Узел не решает, как писать заметку; он вызывает

`self.graph_agent.flush(...)`.

Схема режима `create`:

```
START
  │
  ├─ plan_request            (решаем, нужен ли LLM для выбора движков)
  ├─ choose_engines          (LLM выбирает primary/fallback)
  ├─ finalize_plan           (SearchRouter валидирует и сохраняет план)
  ├─ search                  (primary + fallback, дедупликация)
  ├─ validate                (LLM проверяет тему и требования)
  │
  ├─ если мало статей: suggest_queries → search → validate (до 3 раундов)
  │
  ├─ rank_select             (наукометрия + elbow cutoff)
  ├─ expand_citations        (references/citations для текущей статьи)
  ├─ synthesize              (summary, findings, methodology, limitations, concepts)
  │
  ├─ если --deep: fetch_pages → ground_concepts
  │
  ├─ commit_paper            (статья попадает в base)
  │
  ├─ следующая статья: снова expand_citations → ... → commit_paper
  │
  ├─ rebuild_graph           (концепты, веса, рёбра)
  ├─ define_concepts         (определения концептов батчами)
  ├─ bibliography            (форматированная библиография)
  ├─ link_network            (типизированные связи)
  ├─ overview                (обзорный синтез)
  ├─ flush                   (запись vault)
  └─ audit                   (QA)
END
```

Схема режима `update` короче: после `validate` уже известны новые статьи, поэтому сразу идёт `expand_citations`, затем синтез и commit, а в конце — `save_state` и `audit`. Существующие определения концептов не перезаписываются.

3.4. ResearchState: что передаётся между узлами

`ResearchState` (`lit_graph/agent/graph_state.py`) — это `TypedDict`, в котором лежат только JSON-сериализуемые значения. Ключевые поля:

Поле	Смысл
`run_id`, `mode`	Идентификатор запуска и режим `create`/`update`
`topic`, `requirements`, `directive`	Директива пользователя
`max_papers`, `deep_scan`	Квота и режим PDF
`base`	Вся литературная база как dict
`queries`, `used_queries`, `round`	Поисковые раунды
`candidates`, `valid_candidates`, `selected`	Кандидаты и выбранные статьи
`seen_ids`	Что уже видели, чтобы не дублировать
`current_paper`, `pages`	Текущая статья и страницы PDF
`graph_stats`, `qa_report`	Результаты графа и QA
`activity`	Журнал hand-off с reducer-ом

`activity` использует `Annotated[..., merge_activity]`: каждый узел добавляет событие, а LangGraph склеивает списки. Именно этот журнал потом превращается в `_Agent_Activity.md`.

Runtime-объекты (LLM-клиент, vault, агенты) не лежат в состоянии. Они живут на объекте workflow. Это важно: состояние должно быть сериализуемым, иначе checkpoint и `_state.json` сломаются.

3.5. MemorySaver и `_state.json`

В проекте два уровня состояния:

1. **MemorySaver** — checkpoint LangGraph на время одного запуска. Он позволяет графу хранить промежуточные значения и переживать шаги.
2. **_state.json** — durable-хранилище между запусками. Оно лежит в vault и содержит `LiteratureBaseState`: статьи, концепты, рёбра, библиографию, `search_plan`, `activity`.

`MemorySaver` не заменяет `_state.json`. Если процесс упал, `_state.json` останется; если запуск завершён, именно он позволяет `--update` продолжить базу.

Запись `_state.json` атомарная: сначала пишется `.json.tmp`, затем файл заменяется. Это защищает от ситуации, когда прерванный запуск оставил битый JSON.

3.6. Инварианты архитектуры

Эти правила важно соблюдать при доработке:

- специалист не пишет данные другого специалиста напрямую;
- все промпты живут в `llm/prompts.py`;
- все записи vault идут через `GraphAgent`/`ObsidianVaultManager`;
- QA не мутирует состояние, только читает и сообщает;
- узел возвращает только те ключи `ResearchState`, которые он действительно изменил;
- каждый запуск заканчивается сохранённым `_state.json`;
- новый агент добавляется ролью, сервисом, узлом и тестом, а не правкой «на месте» в середине pipeline.

3.7. Что происходит при ошибке

- Ошибка одного узла логируется и пробрасывается: workflow не должен молча продолжать с битым состоянием.
- Ошибка одного абстракта в бенчмарке не роняет весь прогон: она попадает в `failures`, а бенчмарк идёт дальше.
- Ошибка сети в retrieval не роняет запуск: клиент делает retry/backoff, а кандидат может быть пропущен.
- Ошибка JSON в LLM-ответе пытается восстановиться санитайзером, затем retry, затем fallback.
- QA-ошибки не отменяют запись vault, но попадают в отчёт и CLI.

Именно эта комбинация «жёсткие инварианты + мягкая деградация» делает систему пригодной для реальных запусков, где сеть и локальная модель ведут себя

неидеально.

4. Подробный разбор агентов

В этом разделе каждый агент описан по одной схеме: зачем он нужен, какие данные получает, что делает по шагам, что отдаёт, где лежит код и какие у него ограничения.

4.1. LeadAIAgent (`LiteratureAgent`)

Файл: `lit_graph/agent/core.py`.

Роль: Lead AI Engineer.

Смысл: это «продюсер» проекта. Он не ищет статьи и не пишет заметки вручную; он собирает workflow, запускает его, печатает прогресс и сохраняет итог.

Что получает

- `base_id`, `topic`, `requirements`, `citation_format`;
- путь к vault;
- `max_papers`, `deep_scan`;
- объекты `RetrievalAgent`, `PromptAgent`, `EvaluationAgent`;
- LLM-провайдер и поисковый движок.

Что делает при create

1. Открывает vault и создаёт `GraphAgent`.
2. Создаёт пустой `LiteratureBaseState` с `created_at`/`updated_at`.
3. Формирует начальный `ResearchState`: режим `create`, тема, требования, `queries=[topic]`, пустые кандидаты, `seen_ids=[]`.
4. Собирает `LiteratureWorkflow`, компилирует граф с `MemorySaver`.
5. Запускает `graph.stream(...)` и обновляет прогресс-бар по имени узла.
6. Забирает финальное состояние через `graph.get_state(config).values`.
7. Преобразует `final["base"]` обратно в `LiteratureBaseState`.
8. Печатает таблицу выбранных статей: ранг, название, цитирования, score, weight.
9. Печатает итоговую панель: vault, число источников, концептов, связей.
10. Показывает QA-отчёт.

Что делает при update

1. Биндит существующий vault к `GraphAgent`.
2. Обновляет `updated_at`.
3. Формирует `combined_topic = state.topic + directive`.
4. Запускает workflow в режиме `update` с `queries=[directive]`.
5. Считает, сколько статей добавилось.
6. Показывает QA.

Ключевая деталь: Lead не удаляет старые статьи и не перезаписывает

существующие определения. Он добавляет новое, а GraphAgent сохраняет

старое.

Методы

- `initialize_base(...)` — создать базу.
- `update_base(...)` — расширить базу.
- `flush_to_vault(...)` — принудительно записать vault (используется в CLI).
- `team_table()` — таблица ролей для `litgraph --team`.
- `_run_graph(...)` — общая сборка/запуск workflow.
- `_display_plan(...)` — печать выбранного поискового плана.
- `_report_qa()` — краткий отчёт QA в консоль.

Ограничения

- Lead не валидирует научное качество сам; это делает Prompt/QA.
- Lead не гарантирует, что статей ровно `max_papers`: validation и elbow могут выбрать меньше.
- Lead не спасает от плохой модели: если LLM стабильно ошибается, это видно в QA и метриках, но автоматически не исправляется.

4.2. RetrievalAgent

Файл: `lit_graph/agent/retrieval_agent.py`, клиенты в `lit_graph/search/`.

Роль: Data / IR Engineer.

Смысл: найти кандидатов, не потерять метаданные, не натащить дубликатов и

подготовить данные для валидации и ранжирования.

Каталог движков

`search/tools.py` описывает движки как данные:

- ключ (`semantic_scholar`, `arxiv`, `web`);
- название и краткое описание;
- сильные и слабые стороны;
- лучшие сценарии.

`catalog()` возвращает список для CLI `--tools`; `catalog_prompt()` — текст

для LLM, чтобы модель выбирала движки не «из головы», а из описаний.

plan_request

`plan_request(task, requirements, context)` решает, нужен ли LLM для выбора

движков. Если пользователь передал `--engines`, LLM не нужен: план строится

вручную. Если `--no-router`, используется эвристика. Иначе возвращается

`needs_llm=True`, и workflow идёт в `choose_engines`.

finalize_plan

`finalize_plan(decision, task, requirements, source)` вызывает `SearchRouter`:

1. нормализует ключи движков;
2. отбрасывает неизвестные;
3. ограничивает число primary (`SEARCH_ROUTER_MAX_PRIMARY`);
4. подбирает fallback, если модель его не дала;
5. сохраняет rationale и source.

Если ответ модели невалиден, router строит эвристический план. План записывается в `base.search_plan` и печатается пользователю.

search

`search(query, top_k, engines, fallback_engines, min_results)`:

1. ищет в primary-движках;
2. если результатов меньше `min_results`, добавляет fallback;
3. объединяет метаданные;
4. дедуплицирует по DOI/URL/названию;
5. возвращает кандидатов с `source_engine`.

В workflow поиск повторяется до 3 раундов. Если статей мало, LLM генерирует sub-queries, и цикл `search → validate` повторяется. `seen_ids` не даёт бесконечно возвращать одних и тех же кандидатов.

merge_metadata

`merge_metadata(existing, incoming)` аккуратно сливает записи: не затирает непустые поля пустыми, дополняет отсутствующие. Это важно, потому что arXiv может дать abstract, а Semantic Scholar — citation counts.

expand_citations

`expand_citations(paper_id, limit)` получает references и citations. В workflow они превращаются в рёбра `cites` и сохраняются в `Paper`:

- `referenced_paper_titles`;
- `citing_paper_titles`.

Внешние работы, которых нет в базе, позже получат stub-заметки.

fetch_pages

`fetch_pages(doi_or_url, pdf_url)` скачивает open-access PDF, извлекает текст постранично и возвращает `{page: text}`. Используется только в `--deep`.

Если PDF нет или он защищён, deep-режим деградирует: grounding пропускается, статья всё равно попадает в базу.

Кэш и retry

`utils/http.py` и `utils/cache.py` дают:

- таймауты;

- retry/backoff для 429/5xx;
- дисковый кэш поисков и цитирований;
- TTL (`CACHE_TTL_HOURS`);
- отключение через `--no-cache`.

Ограничения

- Качество зависит от внешних API; при их недоступности база будет меньше.
- Open web шумный; его спасает валидация и ранжирование.
- Citation counts могут быть неполными или устаревшими.
- Дедупликация по названию может склеить разные работы с одинаковым названием; поэтому DOI/URL имеют приоритет.

4.3. PromptAgent и LLM-слой

Файлы: `lit_graph/agent/prompt_agent.py`, `validator.py`,

`synthesizer.py`, `lit_graph/llm/`.

Роль: LLM Engineer.

Смысл: превратить «сырой» текст статьи в структурированные данные и

определения, устойчиво к плохому JSON и ограничениям локальной модели.

Провайдеры

`llm/base.py` задаёт интерфейс, `providers.py` — реализацию для

OpenAI-совместимых API. Поддерживаются DeepSeek, OpenAI, Groq, OpenRouter,

Ollama, vLLM. Переключение — через `.env`:

```
LLM_API_KEY=ollama
LLM_BASE_URL=http://localhost:11434/v1
LLM_MODEL=qwen3.5:4b
```

Провайдер отвечает за таймауты, повторы, температуру и опциональный кэш

ответов по хэшу промпта.

Валидация

`LiteratureValidator.validate_candidate(paper, topic, requirements)` делает

один консолидированный LLM-проход:

- тема релевантна?
- требования выполнены?
- какова научная ценность?

Результат пишется в `Paper`:

- `topic_relevance_score`, `topic_relevance_reason`;
- `requirements_met`, `requirements_reason`;
- `scientific_value_score`.

Кандидат, не прошедший порог, не попадает в выборку, но причина остаётся в

логах/состоянии для отладки.

Синтез статьи

`LiteratureSynthesizer.synthesize_paper_content(paper)` вызывает `paper_synthesis_prompt` и заполняет:

- `summary_en` — executive summary;
- `key_findings_en` — ключевые находки;
- `methodology_en` — методология;
- `limitations_en` — ограничения;
- `extracted_concepts` — концепты (до `MAX_CONCEPTS_PER_PAPER`);
- `extracted_concepts_all` — все концепты до усечения, чтобы повторная каноникализация при `--update` не теряла идеи.

Промпт v2 просит 6–8 конкретных концептов, явно упомянутых в абстракте. Это дало заметный прирост F1 на SciERC (см. раздел 7).

JSON-устойчивость

`_sanitize_and_parse_json`:

1. убирает `<think>`/`<thought>`-блоки;
2. снимает markdown-обёртку ```json;
3. пытается `json.loads`;
4. если не вышло — ищет самый большой валидный объект/массив в тексте;
5. при пустом результате логирует предупреждение.

Это не «делает модель умнее», но спасает от типичных локальных моделей, которые добавляют комментарии или обрезают JSON.

Батчинг

- `CONCEPT_BATCH_SIZE` — сколько концептов определять за один запрос.
- `BIBLIOGRAPHY_BATCH_SIZE` — сколько источников форматировать за раз.

Батчинг нужен, чтобы ответ не обрезался по лимиту токенов. Без него большие базы получали бы половину пустых определений.

Определения концептов

`synthesize_all_concept_hubs_batch(state)` берёт концепты без определения и просит модель написать 2-предложения. Определения сохраняются в `ConceptHub.summary_en`. При `--update` существующие определения не перезаписываются.

Библиография, связи, обзор

- `harmonize_master_bibliography(state)` — единый формат цитат;
- `link_paper_network(state)` — типизированные связи между статьями;
- `build_overall_synthesis(state)` — общий обзор;

- `ground_concepts_to_pages(paper, pages)` — привязка концептов к страницам PDF в deep-режиме.

Ограничения

- Локальные модели 3–4B ограничены: они путают термины и иногда добавляют лишние концепты.
- JSON-санитайзер восстанавливает структуру, но не факты.
- Промпт не заменяет эксперта: summary и findings нужно проверять.
- Определения концептов могут быть общими, если в базе мало контекста.

4.4. GraphAgent

Файлы: `lit_graph/agent/graph_agent.py`, `concepts.py`, `storage/`.

Роль: Graph & Knowledge Engineer.

Смысл: превратить состояние в связный Obsidian-vault и не потерять

пользовательские данные.

rebuild_graph

`rebuild_graph(state, papers)`:

1. собирает концепты из статей;
2. канонизирует имена;
3. считает веса и aggregated score;
4. строит `paper → concept` и `concept → paper` связи;
5. сохраняет `state.concepts` и `state.edges`;
6. возвращает статистику: сколько концептов/рёбер.

Каноникализация

`concepts.py`:

- `concept_key(name)` — ключ сравнения: lowercase, без пунктуации, parenthetical, с раскрытием известных акронимов и singularization;
- `display_name(name)` — человекочитаемое имя: smart title case, сохранение акронимов и mixed-case;
- `choose_display_names(names)` — выбирает самый частый вариант;
- `dedupe_concepts(names)` — убирает дубли.

Пример: `GNNs` → ключ `graph neural network`, display `Graph Neural Network`.

Веса

Вес концепта зависит от числа упоминаний и наукометрического веса статей.

Нормализация в 0..1 позволяет использовать размер узла в graph view.

`MIN_CONCEPT_PAPERS` отсекает концепты, которые встретились меньше чем в N

статьях; это шумовой фильтр.

flush

`flush(state, activity)`:

1. готовит имена файлов;

2. пишет заметки источников;
3. пишет концепт-хабы;
4. пишет внешние stubs;
5. пишет `_Overview_Synthesis.md`, `_Bibliography.md`, `Formatted_Bibliography.md`, `references.bib`;
6. пишет `_Agent_Activity.md`;
7. сохраняет `_state.json`.

Имена и YAML

`ObsidianVaultManager`:

- `clean_name` убирает запрещённые символы;
- `_truncated_stem` обрезает слишком длинные имена и добавляет хэш;
- `_register` разрешает коллизии;
- `_yaml` экранирует значения, чтобы двоеточия и кавычки не ломали frontmatter.

graph.json

Пресет графа Obsidian устанавливается при первом запуске и **сливается** с

пользовательскими настройками, а не перезаписывает их. Это важно: иначе

каждый запуск сбрасывал бы цвета и фильтры пользователя.

Ограничения

- Каноникализация эвристическая: сложные синонимы могут не слиться.
- Слишком агрессивное слияние может объединить разные сущности.
- Graph view — визуализация, а не доказательство качества; QA проверяет ссылки, но не смысл связей.

4.5. EvaluationAgent

Файлы: `lit_graph/agent/evaluation_agent.py`, `qa/vault_checks.py`.

Роль: QA / Evaluation Engineer.

Смысл: независимо проверить, что vault и состояние не противоречат друг

другу.

audit

`audit(state, vault_path)`:

1. `run_vault_checks(vault_path)` — проверки файлов и ссылок;
2. `run_state_checks(state)` — проверки данных;
3. merge отчётов;
4. логирование summary.

Проверки vault

- `vault_missing` — папки нет;
- `vault_empty` — нет Markdown-файлов;
- `duplicate_stem` — одинаковые имена заметок;

- `missing_frontmatter` — нет YAML-блока;
- `missing_frontmatter_key` — не хватает обязательного ключа;
- `unknown_note_type` — неизвестный `type`;
- `unresolved_link` — `[[ссылка]]` не находит цель;
- `orphan_note` — заметка без входящих/исходящих ссылок;
- `empty_bibliography` — библиография пустая.

Проверки состояния

- `no_papers` — база пустая;
- `missing_summary` — у статьи нет summary;
- `missing_citation` — нет форматированной цитаты;
- `no_concepts` — статья без концептов;
- `orphan_concept_link` — статья ссылается на концепт, которого нет в state;
- `missing_concept_definition` — у концепта нет определения;
- `concept_without_mentions` — концепт без статей;
- `empty_bibliography_state` — пустая библиография в state;
- `empty_overview` — пустой обзор;
- `duplicate_edge` — дубликат ребра;
- `unknown_edge_source` / `unknown_edge_target` — ребро ссылается на неизвестную статью.

Severity

- `error` — критично, обычно ломает целостность;
- `warning` — подозрительно, но база может работать;
- `info` — заметка для диагностики.

`litgraph --check --id ...` печатает отчёт; в CI можно проверять наличие

errors.

Ограничения

- QA проверяет структуру и целостность, а не научную истинность.
- Он не может понять, что статья нерелевантна, если LLM её пропустил.
- Он не заменяет бенчмарк качества извлечения.

5. Сквозные сценарии и данные

5.1. Сценарий create: из темы в vault

Пользователь запускает:

```
litgraph \
  --id "transformer_circuits" \
  --topic "Mechanistic Interpretability and Induction Heads in Transformers" \
  --requirements "Peer-reviewed or high-impact preprints published after 2021" \
  --format "APA 7th" \
  --max-papers 5
```

Что происходит по шагам:

1. **CLI** проверяет аргументы, создаёт/открывает vault, загружает настройки.
2. **Lead** создаёт пустое состояние и запускает LangGraph.
3. **plan_request** решает, нужен ли LLM для выбора движков.
4. **choose_engines** просит модель выбрать primary/fallback из каталога.
5. **finalize_plan** валидирует план и сохраняет `search_plan`.
6. **search** ищет кандидатов, дедуплицирует, помечает `source_engine`.
7. **validate** проверяет тему и требования; кандидаты без прохода отбрасываются.
8. Если статей мало, **suggest_queries** генерирует подзапросы и цикл повторяется (до 3 раундов).
9. **rank_select** сортирует по наукометрическому скору и применяет elbow cutoff.
10. Для каждой выбранной статьи: - **expand_citations** добавляет references/citations; - **synthesize** извлекает summary/findings/methodology/limitations/ concepts; - в `--deep` — **fetch_pages** и **ground_concepts**; - **commit_paper** добавляет статью в `base`.
11. **rebuild_graph** канонизирует концепты и строит рёбра.
12. **define_concepts** пишет определения концептов.
13. **bibliography** собирает цитаты.
14. **link_network** строит типизированные связи.
15. **overview** пишет общий обзор.
16. **flush** записывает vault.
17. **audit** проверяет ссылки, frontmatter, дубли, сирот, целостность.

В конце пользователь видит таблицу выбранных статей и QA-отчёт.

5.2. Сценарий update: продолжить базу

```
litgraph \
  --id "transformer_circuits" \
  --update "Sparse Autoencoders and Superposition in Language Models" \
  --max-papers 2
```

Отличия от create:

- Lead загружает `_state.json` и передаёт существующую базу в workflow;
- `queries=[directive]`, а тема = исходная тема + директива;
- после validation сразу идёт ingestion, без повторного rank_select;
- новые статьи добавляются к старым;

- существующие определения концептов не перезаписываются;
- в конце `save_state` и `audit`.

Это позволяет расширять базу неделями, не теряя предыдущую работу.

5.3. Сценарий deep: PDF и grounding

```
litgraph --id "transformer_circuits" --update "Induction heads" --deep
```

В deep-режиме после синтеза:

1. `fetch_pages` скачивает open-access PDF;
2. `ground_concepts` ищет упоминания концептов по страницам;
3. в `ConceptMention` появляются `page_number` и `context_snippet`;
4. в концепт-хабе появляются ссылки вида «источник, страница, цитата».

Если PDF недоступен, статья всё равно обрабатывается по абстракту. Deep не является обязательным для работы.

5.4. Сценарий check, list, delete

```
litgraph --list
litgraph --check --id "transformer_circuits"
litgraph --delete --id "transformer_circuits"
```

- `--list` показывает существующие базы.
- `--check` запускает QA и печатает issues.
- `--delete` удаляет базу (осторожно: это необратимо).

5.5. Модель данных

Основные модели лежат в `lit_graph/models.py`.

Paper

`Paper` — статья. Ключевые поля:

- идентификация: `id`, `title`, `authors`, `year`, `venue`, `doi_or_url`, `pdf_url`, `source_engine`;
- наукометрия: `citation_count`, `influential_citation_count`, `citation_velocity`, `influential_ratio`, `objective_scientific_score`, `normalized_weight`, `is_elbow_selected`;
- валидация: `topic_relevance_score`, `topic_relevance_reason`, `requirements_met`, `requirements_reason`, `scientific_value_score`;
- синтез: `summary_en`, `key_findings_en`, `methodology_en`, `limitations_en`, `extracted_concepts`, `extracted_concepts_all`, `concept_mentions`;
- связи: `referenced_paper_titles`, `citing_paper_titles`, `related_paper_titles`;
- библиография: `formatted_citation`.

`extracted_concepts` — усечённый список до `MAX_CONCEPTS_PER_PAPER`;

`extracted_concepts_all` — полный, чтобы повторная каноникализация не теряла идеи.

ConceptHub

`ConceptHub` — концепт:

- `name` — display-имя;
- `summary_en` — определение;
- `aggregated_score` — суммарный вес;
- `normalized_weight` — вес 0..1 для визуализации;
- `mentions` — список `ConceptMention`.

ConceptMention

`ConceptMention` — упоминание концепта в статье:

- `concept_name`;
- `paper_title`;
- `page_number`;
- `context_snippet`.

GraphEdge

`GraphEdge` — ребро:

- `source_title`;
- `target_title`;
- `relation_type` (`cites`, `relates_to` и т.д.).

`LiteratureBaseState.add_edge` не добавляет дубликаты.

LiteratureBaseState

`LiteratureBaseState` — вся база:

- `base_id`, `topic`, `requirements`, `citation_format`;
- `overall_summary_en`;
- `papers: Dict[str, Paper]`;
- `concepts: Dict[str, ConceptHub]`;
- `edges: List[GraphEdge]`;
- `bibliography_en`;
- `search_plan`;
- `created_at`, `updated_at`.

5.6. `_state.json`

`_state.json` — JSON-представление `LiteratureBaseState`. Он лежит в корне vault. Пример (сокращённо):

```
{
  "base_id": "transformer_circuits",
  "topic": "Mechanistic Interpretability...",
  "requirements": "Peer-reviewed...",
  "citation_format": "APA 7th",
  "papers": {
    "2401.00001": {
      "id": "2401.00001",
```

```
      "title": "Example Paper",
      "year": 2024,
      "citation_count": 42,
      "extracted_concepts": ["Induction Head", "Transformer"],
      "source_engine": "arxiv"
    }
  },
  "concepts": {
    "Induction Head": {
      "name": "Induction Head",
      "summary_en": "A circuit that...",
      "mentions": []
    }
  },
  "edges": [],
  "search_plan": {
    "primary": ["arxiv", "semantic_scholar"],
    "fallback": ["web"],
    "source": "llm"
  }
}
```

Запись атомарная: `.json.tmp` → replace. Если процесс упал, старый файл

остаётся целым.

5.7. Структура vault

```
vaults/transformer_circuits/
├── Formatted_Bibliography.md
├── _Bibliography.md
├── _Overview_Synthesis.md
├── _Agent_Activity.md
├── _state.json
├── references.bib
├── Sources/
│   ├── <paper title>.md
│   └── <external reference>.md
└── Concepts/
    └── <concept name>.md
```

Заметка источника

Frontmatter:

```
id: "2401.00001"
title: "Example Paper"
authors: ["A. Author"]
year: 2024
venue: "arXiv"
source_engine: "arxiv"
citation_count: 42
citation_velocity: 10.5
objective_score: 12.3
node_weight: 0.71
type: literature_note
```

Секции:

- Formatted Citation;
- Executive Summary;
- Key Findings;

- Methodology & Limitations;
- Knowledge Graph Connections;
- Typed Graph Relations (если есть);
- Abstract.

Концепт-хаб

Frontmatter:

```
concept: "Induction Head"
aggregated_score: 4.2
node_weight: 0.83
type: concept_hub
```

Секции:

- Concept Definition & Consensus;
- Grounded Citations & Page References.

Внешняя заглушка

```
title: "External Paper"
type: external_reference
```

Секция «Cited within this vault» со ссылками на источники.

Хаб-файлы

- `_Overview_Synthesis.md` — обзор, индекс, типизированные связи;
- `_Bibliography.md` — интерактивная библиография с `[[links]]`;
- `Formatted_Bibliography.md` — чистый список для копирования;
- `references.bib` — BibTeX;
- `_Agent_Activity.md` — журнал: кто, что, когда, сколько заняло.

5.8. Почему vault, а не база данных

Vault выбран по трём причинам:

1. **Прозрачность.** Каждую заметку можно открыть и поправить руками.
2. **Локальность.** Нет облачной зависимости и vendor lock-in.
3. **Obsidian.** Граф, обратные ссылки и поиск уже есть; не нужно писать UI.

Минус — нет транзакций и строгой схемы. Поэтому в проекте есть QA, детерминированные имена, YAML-экранирование и `_state.json` как источник правды.

5.9. Что происходит при повторном запуске

- `_state.json` загружается;
- существующие статьи не дублируются по `id`;
- существующие концепты переиспользуются;
- определения не перезаписываются, если уже есть;

- новые статьи добавляются;
- граф перестраивается, но пользовательские настройки `graph.json` сохраняются;
- QA проверяет итог.

Именно это делает базу «живой», а не одноразовым экспортом.

6. Как устроены поиск, LLM и граф

Этот раздел — «начинка» трёх самых важных подсистем. Здесь меньше слов о роли агентов и больше о том, как именно принимаются решения.

6.1. Поиск и выбор движков

Каталог как данные

В `search/tools.py` каждый движок описан структурой:

- `key` — `semantic_scholar`, `arxiv`, `web`;
- `name` — человекочитаемое имя;
- `description` — что это;
- `strengths` — чем хорош;
- `weaknesses` — где ошибается;
- `best_for` — типовые задачи.

Модель получает этот каталог текстом и возвращает JSON:

```
{
  "primary": ["arxiv", "semantic_scholar"],
  "fallback": ["web"],
  "reason": "Fresh preprints plus citation metadata; web only as a fallback."
}
```

SearchRouter

`search/router.py` не доверяет ответу модели напрямую:

1. проверяет, что ключи известны;
2. нормализует регистр и пробелы;
3. убирает дубликаты;
4. ограничивает primary;
5. гарантирует непустой fallback;
6. сохраняет `source` (`llm`, `manual`, `heuristic`).

Если модель вернула мусор, router строит эвристику. Если пользователь передал `--engines`, LLM вообще не вызывается. Если `--no-router`, используется эвристика.

Почему не «опросить всё»

Опросить все движки сразу — дорого и шумно. Модель выбирает 1–2 primary и fallback, исходя из темы. Например, для свежих preprint-ов лучше arXiv, для цитатной наукометрии — Semantic Scholar, для узких веб-источников — web.

Дедупликация

Кандидаты приходят из разных источников. `merge_metadata` и дедупликация:

- сравнивают DOI;
- сравнивают URL;
- сравнивают нормализованное название;
- сохраняют более полные метаданные;
- не затирают непустые поля пустыми.

Ранжирование

`compute_scientific_metrics(paper)` считает:

```
age = max(1, current_year - year)
velocity = citation_count / age
influential_ratio = (influential_citations + 1) / (citations + 1)
log_volume = log10(citations + 1)
score = min(velocity, 100) * 0.40
      + log_volume * 4.0
      + influential_ratio * 10.0 * 0.25
```

Смысл компонентов:

- **velocity** — свежесть и скорость набора цитирований;
- **log_volume** — общий объём цитирований, сжатый логарифмом, чтобы мега-цитируемые работы не доминировали;
- **influential_ratio** — доля влиятельных цитирований.

Веса (40/35/25 в комментарии, но в коде log-компонент масштабирован на 4.0)

эвристические. Они не откалиброваны на размеченном наборе; это честно

зафиксировано в отчёте.

Elbow cutoff

`find_elbow_cutoff(scores, min_k, max_k)`:

1. сортирует scores по убыванию;
2. считает drops между соседними;
3. считает curvature = drops[i] - drops[i+1];
4. выбирает максимальную положительную кривизну;
5. ограничивает результат `min_k..max_k`.

Идея: в голове списка scores падают медленно, затем резко, затем хвост

выравнивается. «Локоть» — граница головы. Это не строгая статистика, а

эвристика для отсечения низкоимпактного хвоста.

Кэш и retry

`utils/http.py`:

- таймауты;
- retry с backoff на 429/5xx;
- единый User-Agent.

`utils/cache.py`:

- дисковый кэш поисков и цитирований;

- TTL;
- отключение через `--no-cache`.

Кэш важен для воспроизводимости и скорости: повторный запуск не дёргает API заново.

PDF

`search/pdf_extractor.py`:

- проверяет размер;
- извлекает текст постранично;
- ограничивает объём;
- возвращает `{page: text}`.

В deep-режиме текст страниц идёт в grounding-промпт. Если PDF нет, deep деградирует, но не ломает запуск.

6.2. LLM-слой

Провайдеры

`llm/base.py` — интерфейс `generate`/`generate_json`.

`llm/providers.py` — OpenAI-совместимый клиент. Он работает с любым endpoint, который поддерживает `/chat/completions`.

Профили:

```
# Ollama
LLM_API_KEY=ollama
LLM_BASE_URL=http://localhost:11434/v1
LLM_MODEL=qwen3.5:4b

# DeepSeek
LLM_API_KEY=sk-...
LLM_BASE_URL=https://api.deepseek.com
LLM_MODEL=deepseek-chat
```

Промпты

`llm/prompts.py` — единая точка. Основные:

Функция	Назначение
`engine_selection_prompt`	выбрать primary/fallback
`sub_queries_prompt`	сгенерировать подзапросы
`validation_prompt`	тема + требования
`paper_synthesis_prompt`	summary, findings, methodology, limitations, concepts
`concept_definitions_prompt`	определения концептов
`concept_grounding_prompt`	привязка к страницам PDF
`bibliography_prompt`	форматирование цитат
`relations_prompt`	типизированные связи
`review_prompt` / `merge_reviews_prompt`	обзор

Важно: `LiteratureSynthesizer` использует `paper_synthesis_prompt`, поэтому формулировка живёт в одном месте. Раньше был дубликат inline-промпта; его убрали, иначе изменения в `prompts.py` не влияли на бенчмарк.

JSON-санитайзер

Локальные модели часто возвращают:

- `<think>...</think>` перед JSON;
- ``` ``json ... `` ```;
- комментарии;
- обрезанный JSON.

`_sanitize_and_parse_json`:

1. удаляет reasoning-блоки;
2. снимает code fence;
3. пробует `json.loads`;
4. ищет самый большой валидный объект/массив;
5. возвращает `{}` при полной неудаче и логирует.

Это делает пайплайн устойчивым, но не гарантирует правильность фактов.

Батчинг

- `CONCEPT_BATCH_SIZE=12` — определения концептов;
- `BIBLIOGRAPHY_BATCH_SIZE=20` — цитаты.

Батчинг уменьшает риск обрезания ответа и позволяет большим базам обрабатываться частями.

Промпт v2 для концептов

Старый промпт просил JSON с тремя примерами концептов и не задавал количество. Модель выдавала в среднем 5.75 концепта, часто общих.

Новый промпт:

- просит 6–8 конкретных концептов;
- требует, чтобы они были явно упомянуты;
- просит methods/tasks/metrics/materials/datasets/technical terms;
- запрещает generic-слова и дубликаты;
- ставит `extracted_concepts` первым в JSON.

Результат на 20 абстрактах:

- average concepts: 5.75 → 8.0;
- exact F1: 0.349 → 0.422;
- relaxed F1: 0.501 → 0.583;
- content relaxed F1: 0.549 → 0.629.

Цена — время: 1836.9 → 3571.0 секунд на 20 абстрактов (~179 с на абстракт).

Валидация

Один консолидированный вызов проверяет тему и требования. Это дешевле и меньше рассинхронизирует два отдельных прохода. Результат сохраняется в `Paper`; кандидат, не прошедший порог, отбрасывается.

Синтез

`paper_synthesis_prompt` возвращает структуру, а не свободный текст. Это позволяет:

- писать заметку без повторного парсинга;
- передавать концепты в каноникализацию;
- хранить полный и усечённый список концептов;
- повторно канонизировать при `--update`.

6.3. Граф и каноникализация

Зачем каноникализация

LLM возвращает `Transformer`, `Transformers`, `GNN`, `GNNs`, `Graph Neural Networks (GNNs)`. Без слияния это четыре разных узла, и граф теряет смысл.

`concepts.py`:

- `_normalize_whitespace`;
- `_strip_edges`;
- `_singularize_word` с учётом uncountable;
- `_strip_parenthetical` (оставляет описательную часть, если она длиннее аббревиатуры);
- `ABBREVIATIONS` (GNN, LLM, SAE, ViT, MLP, CNN, RNN, NLP, CoT);
- `concept_key` — ключ сравнения;
- `display_name` — человекочитаемое имя;
- `choose_display_names` — самый частый вариант;
- `dedupe_concepts` — уникальные пары `(key, display)`.

Пример:

```
"Graph Neural Networks (GNNs)"
  key:     "graph neural network"
  display: "Graph Neural Network"
```

Веса

Вес концепта зависит от:

- числа упоминаний;
- веса статей, где он встречается;
- агрегированного score.

Нормализация в 0..1 даёт размер узла в graph view. `MIN_CONCEPT_PAPERS`

отсекает концепты, которые встретились меньше чем в N статьях. Это снижает шум, но может потерять редкие, но важные термины.

Запись заметок

`ObsidianVaultManager.write_paper_note`:

- формирует frontmatter;
- добавляет citation, summary, findings, methodology, limitations;
- добавляет связи;
- добавляет typed relations;
- добавляет abstract.

`write_concept_note`:

- frontmatter с `concept`, `aggregated_score`, `node_weight`;
- определение;
- grounded citations с страницами и цитатами.

`write_external_stubs`:

- собирает внешние referenced/citing titles;
- создаёт лёгкие заметки, чтобы ссылки не повисали.

`write_hub_files`:

- `_Overview_Synthesis.md`;
- `_Bibliography.md`;
- `Formatted_Bibliography.md`;
- `references.bib`;
- `_Agent_Activity.md`.

Безопасность имён и YAML

- `clean_name` убирает `/`, `\`, `:`, `*`, `?`, `"`, `<`, `>`, `|`;
- длинные имена обрезаются с хэшем;
- коллизии разрешаются суффиксом;
- `_yaml` экранирует значения;
- `graph.json` сливается, а не перезаписывается.

Что может пойти не так

- Слишком агрессивная каноникализация объединит разные сущности.
- Слишком слабая — оставит дубли.
- `MIN_CONCEPT_PAPERS` может удалить редкий, но важный концепт.
- Graph view красив, но не доказывает семантическую правильность связей.

Именно поэтому QA и бенчмарк существуют отдельно от «выглядит красиво».

7. QA, тесты и бенчмарк

7.1. Зачем отдельный QA-агент

LLM и внешние API ошибаются. Если писать vault без проверки, можно получить:

- `[[ссылки]]`, которые никуда не ведут;
- заметки без обязательного frontmatter;
- дубликаты имён;
- статьи, которые ссылаются на концепты, которых нет;
- пустую библиографию;
- «сиротские» заметки, до которых нельзя дойти по ссылкам.

QA-агент не мутирует данные. Он читает vault и состояние, собирает issues с severity и отдаёт отчёт. Это позволяет использовать `litgraph --check` в CI и на демонстрации.

7.2. Проверки vault

`qa/vault_checks.py` проходит по всем `.md` вне `.obsidian` и `.trash`:

- **vault_missing** — папки нет;
- **vault_empty** — нет Markdown-файлов;
- **duplicate_stem** — одинаковые имена заметок;
- **missing_frontmatter** — нет YAML-блока;
- **missing_frontmatter_key** — не хватает обязательного ключа;
- **unknown_note_type** — неизвестный `type`;
- **unresolved_link** — `[[ссылка]]` не находит цель;
- **orphan_note** — заметка без входящих/исходящих ссылок;
- **empty_bibliography** — в библиографии нет ссылок.

Обязательные ключи по типам:

Тип	Ключи
`literature_note`	`title`, `type`
`concept_hub`	`concept`, `type`
`external_reference`	`title`, `type`
`synthesis_hub`	`topic`, `type`
`master_bibliography`	`citation_format`, `type`
`agent_activity`	`type`

Статистика отчёта: `notes`, `sources`, `concepts`, `external_refs`, `links`, `unresolved_links`, `orphans`.

7.3. Проверки состояния

`run_state_checks(state)` проверяет данные, а не файлы:

- **no_papers** — база пустая;
- **missing_summary** — нет summary;
- **missing_citation** — нет formatted citation;
- **no_concepts** — статья без концептов;
- **orphan_concept_link** — статья ссылается на отсутствующий hub;
- **missing_concept_definition** — у концепта нет определения;
- **concept_without_mentions** — концепт без статей;
- **empty_bibliography_state** — пустая библиография;
- **empty_overview** — пустой обзор;
- **duplicate_edge** — дубликат ребра;
- **unknown_edge_source** / **unknown_edge_target** — ребро ссылается на неизвестную статью.

Severity:

- `error` — критично;
- `warning` — подозрительно, но база может работать;
- `info` — диагностика.

`EvaluationAgent.audit` объединяет vault + state отчёты и логирует summary.

7.4. Тесты и CI

В проекте 55 тестов. Они покрывают:

- pipeline и агентов;
- персистентность `_state.json`;
- каноникализацию концептов;
- имена заметок;
- QA;
- батчинг;
- провайдеры и JSON-санитайзер;
- поисковый роутер;
- ранжирование;
- бенчмарк.

Команды:

```
pytest -q
ruff check lit_graph tests benchmarks
```

CI:

- GitHub Actions;
- Python 3.10, 3.11, 3.12, 3.13;
- `ruff` валит сборку при ошибках;
- `pytest` на push/PR.

CI не делает реальных сетевых вызовов и не запускает локальную LLM: E2E

проверяется локально, а CI ловит регрессии структуры и логики.

7.5. SciERC: что это

SciERC (Luan et al., EMNLP 2018) — стандартный бенчмарк для scientific knowledge-graph construction. В нём 500 абстрактов научных статей с аннотациями:

- сущности 6 типов: Task, Method, Metric, Material, Generic, OtherScientificTerm;
- типизированные отношения;
- coreference-кластеры.

Это не «граф вообще», а именно извлечение научных сущностей и связей из текста. Поэтому SciERC релевантен LitGraphAgent, который тоже строит граф из научных статей.

7.6. Что именно измеряет наш harness

`benchmarks/scierc_benchmark.py`:

1. загружает `.txt` + `.ann` из SciERC;
2. выбирает `limit` абстрактов с `seed`;
3. для каждого вызывает production `PromptAgent.synthesize_paper`;
4. сравнивает `extracted_concepts` с gold-упоминаниями;
5. считает concept P/R/F1 и paper→concept edges;
6. сохраняет JSON + JSONL checkpoint;
7. умеет resume после прерывания;
8. пишет failures, не роняя весь прогон.

Что **не** измеряется:

- типизированные отношения SciERC;
- coreference;
- span boundaries;
- relation extraction.

Причина: пайплайн не предсказывает типы отношений SciERC. Считать relation F1 было бы фикцией. Это ограничение явно указано в README и отчёте.

7.7. Exact, relaxed и content-only

Exact — нормализованное равенство строк:

- lowercase;
- singularization;
- удаление пунктуации;
- удаление артиклей.

Relaxed — token Jaccard ≥ 0.5. Допускает модификаторы и сокращения: `word sense disambiguation (WSD) systems` и `word sense disambiguation` считаются совпадением.

Content-only — gold-сущности без типа Generic. Это отдельный сигнал:

Generic-слова (`system`, `method`, `approach`) часто шумят, и без них

метрика показывает содержательные концепты.

Метрики считаются micro: TP/FP/FN агрегируются по всем абстрактам, затем

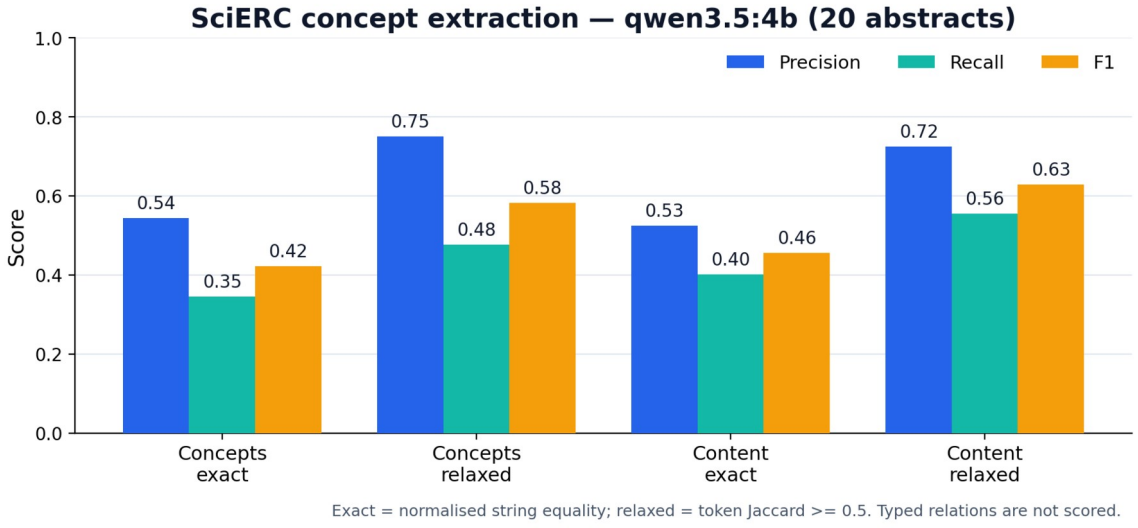
считаются P/R/F1. Это стандартно для extraction, но нужно явно указывать.

7.8. Результаты на 20 абстрактах

Протокол:

- 20 абстрактов;
- `seed=42`;
- локальные модели через Ollama;
- production-вызов `PromptAgent.synthesize_paper`;
- 0 падений.

Модель	Сигнал	Precision	Recall	F1
`qwen2.5:3b`	Concepts, exact	0.538	0.254	0.345
`qwen2.5:3b`	Concepts, relaxed	0.723	0.341	0.464
`qwen2.5:3b`	Content, relaxed	0.706	0.402	0.512
`qwen3.5:4b`, prompt v1	Concepts, exact	0.556	0.254	0.349
`qwen3.5:4b`, prompt v1	Concepts, relaxed	0.800	0.365	0.501
`qwen3.5:4b`, prompt v1	Content, relaxed	0.774	0.426	0.549
`qwen3.5:4b`, prompt v2	Concepts, exact	0.544	0.345	0.422
`qwen3.5:4b`, prompt v2	Concepts, relaxed	0.750	0.476	0.583
`qwen3.5:4b`, prompt v2	Content, relaxed	0.725	0.555	0.629



Метрики SciERC для qwen3.5:4b, prompt v2

Среднее число концептов:

- `qwen2.5:3b` — 5.95;
- `qwen3.5:4b` prompt v1 — 5.75;
- `qwen3.5:4b` prompt v2 — 8.0;
- gold — 12.6.

Время:

- `qwen2.5:3b` — 252.7 с;
- `qwen3.5:4b` prompt v1 — 1836.9 с;
- `qwen3.5:4b` prompt v2 — 3571.0 с (~179 с на абстракт).

7.9. Что дал prompt v2

Старый промпт не задавал количество концептов и показывал три примера. Модель выдавала ~5.75 концепта, часто общих.

Новый промпт:

- просит 6–8 концептов;
- требует, чтобы они были явно упомянуты;
- покрывает methods/tasks/metrics/materials/datasets/technical terms;
- запрещает generic и дубликаты;
- ставит `extracted_concepts` первым в JSON.

Прирост:

- exact F1: +0.073;
- relaxed F1: +0.082;
- content relaxed F1: +0.080.

Цена: почти вдвое дольше, потому что модель генерирует больше концептов и отвечает стабильно длиннее. Для исследовательской базы это приемлемый trade-off; для интерактивного сценария — нет.

7.10. Опорные точки

Точка	Exact F1	Relaxed F1	Content relaxed F1	Смысл
Пустое предсказание	0.000	0.000	0.000	нижняя граница
`qwen2.5:3b`	0.345	0.464	0.512	меньшая модель
`qwen3.5:4b`, prompt v1	0.349	0.501	0.549	предыдущий промпт
`qwen3.5:4b`, prompt v2	0.422	0.583	0.629	текущий результат
Gold oracle	1.000	1.000	1.000	верхняя граница

Внешний ориентир: SciERC SOTA (SURE, COLING 2025):

- entity F1 73.4;
- relation F1 56.9;

- strict relation F1 44.3.

Это span-level NER/RE, а не извлечение канонических концептов. Прямое сравнение некорректно: у нас другая постановка и другой матчинг.

7.11. Ограничения бенчмарка

- 20 абстрактов — мало для статистической значимости.
- Один seed.
- Одна модель (плюс меньшая 3B для сравнения).
- Оценены только концепты, не отношения.
- Нет end-to-end метрики поиска (precision@k, nDCG).
- Нет оценки качества итогового обзора.
- LLM недетерминирована; один прогон не равен среднему.
- Exact — нижняя граница из-за каноникализации.
- SOTA SciERC — другая задача.

До КП3 имеет смысл:

- 100 абстрактов;
- 3 seed;
- 2–3 модели;
- triple-level F1, если появятся типы отношений;
- coverage, orphan rate, unresolved link rate;
- precision@k/nDCG для поиска;
- p95 latency и cost per run.

7.12. Как читать эту таблицу

1. Смотрите на **content relaxed F1** как на основной сигнал качества концептов.
2. Смотрите на **exact F1** как на нижнюю границу: каноникализация мешает точному совпадению.
3. Сравнивайте только строки с одинаковым протоколом (модель, seed, prompt).
4. Не сравнивайте наш concept F1 с SciERC entity/relation F1 напрямую.
5. Помните, что 20 абстрактов — regression sample, а не лидерборд.

8. Метрики с нуля

Этот раздел можно читать отдельно от проекта. Он объясняет, что означают числа в таблицах и почему нельзя сравнивать метрики разных задач.

8.1. Задача извлечения концептов

У нас есть абстракт статьи. Есть **gold** — эталонный список концептов, размеченный людьми. Есть **prediction** — список, который вернула система.

Нужно ответить на два вопроса:

1. Насколько найденное правильное? — **precision**.
2. Насколько полно найдено эталонное? — **recall**.

8.2. TP, FP, FN и TN

Для каждого концепта есть четыре возможности:

	Система сказала «есть»	Система сказала «нет»
В gold есть	TP (true positive)	FN (false negative)
В gold нет	FP (false positive)	TN (true negative)

В извлечении концептов TN почти не используется: пространство всех возможных концептов огромно, и «правильно не назвать» бессмысленно считать. Поэтому оперируют TP, FP, FN.

8.3. Precision

```
Precision = TP / (TP + FP)
```

«Из всего, что система назвала, какая доля действительно правильная».

Пример: система вернула 8 концептов, 5 из них есть в gold.

```
Precision = 5 / 8 = 0.625
```

Precision падает, когда система добавляет лишние, «шумные» концепты.

8.4. Recall

```
Recall = TP / (TP + FN)
```

«Из всего, что было в gold, какую долю система нашла».

В gold 10 концептов, найдено 5.

```
Recall = 5 / 10 = 0.5
```

Recall падает, когда система пропускает концепты. В нашей задаче recall часто ниже precision, потому что модель возвращает 6–8 концептов, а gold содержит в среднем 12.6.

8.5. F1

```
F1 = 2 * Precision * Recall / (Precision + Recall)
```

F1 — гармоническое среднее. Оно сильнее наказывает за дисбаланс: если precision 1.0, а recall 0.1, F1 будет низким.

Пример:

```
P = 0.625, R = 0.5
F1 = 2 * 0.625 * 0.5 / (0.625 + 0.5) = 0.625 / 1.125 = 0.556
```

F1 удобен, когда нужно одно число, но он скрывает, что именно происходит: система может быть очень точной, но находить мало, или наоборот.

8.6. Почему не accuracy

Accuracy = (TP + TN) / все. В извлечении концептов TN огромно и не имеет смысла: «правильно не назвать случайное слово» — не полезная работа. Поэтому accuracy здесь не используют.

8.7. Exact и relaxed

Exact — строки должны совпасть после нормализации:

- lowercase;
- единственное число;
- без пунктуации;
- без артиклей.

Пример:

```
gold:       "word sense disambiguation (WSD) systems"
prediction: "word sense disambiguation"
exact:      нет (строки разные)
relaxed:    да (Jaccard ≥ 0.5)
```

Relaxed — token Jaccard:

```
Jaccard = |пересечение токенов| / |объединение токенов|
```

Если Jaccard ≥ 0.5, считаем совпадением. Это допускает сокращения и

модификаторы, но может склеить близкие, но разные концепты.

В отчёте exact всегда ниже relaxed, потому что каноникализация меняет строки.

Exact — нижняя граница, relaxed — более честная оценка смысла.

8.8. Micro и macro

Micro — складываем TP/FP/FN по всем примерам и считаем P/R/F1 один раз.

Доминируют большие статьи.

Macro — считаем P/R/F1 для каждой статьи, затем усредняем. Все статьи

равны, даже маленькие.

В нашем бенчмарке используется micro. Это стандартно для extraction, но

нужно указывать явно: micro и macro могут дать разные числа.

Пример:

Статья	TP	FP	FN
A	5	3	5
B	1	1	9

Micro:

```
TP=6, FP=4, FN=14
P = 6/10 = 0.6
R = 6/20 = 0.3
F1 = 0.4
```

Macro:

```
A: P=0.625, R=0.5, F1=0.556
B: P=0.5, R=0.1, F1=0.167
macro F1 = (0.556 + 0.167)/2 = 0.361
```

Числа отличаются. Поэтому в отчёте важно писать, какой режим используется.

8.9. Content-only

Generic-сущности (`system`, `method`, `approach`, `algorithm`) часто являются

шумом. Content-only исключает их из gold и считает метрику по содержательным

концептам. Это не «улучшение модели», а другой срез оценки.

Если модель предсказывает generic-слова, в content-only они становятся FP и

снижают precision. Поэтому content-only дисциплинирует.

8.10. MRR

MRR (Mean Reciprocal Rank) используется в задачах ранжирования: для каждого

запроса правильный ответ имеет ранг; берём `1/rank` и усредняем.

Пример: правильный ответ на позициях 1, 2, 4.

```
MRR = (1/1 + 1/2 + 1/4) / 3
    = (1 + 0.5 + 0.25) / 3
    = 0.583
```

MRR хорош, когда важен первый правильный ответ. Он сильно вознаграждает ранг 1 и почти не чувствует разницу между 100 и 101.

8.11. Hits@k

Hits@k — доля запросов, где правильный ответ попал в top-k.

```
Hits@10 = число запросов с rank ≤ 10 / все запросы
```

Пример: 100 запросов, в 60 правильный ответ в top-10 → Hits@10 = 0.6.

Hits@k не учитывает, на каком именно месте внутри top-k; для этого есть MRR и nDCG.

8.12. nDCG

nDCG учитывает и порядок, и градацию релевантности.

```
DCG@k = Σ (2^rel_i - 1) / log2(i + 1)
nDCG@k = DCG@k / IDCG@k
```

`IDCG` — DCG идеального порядка. nDCG ∈ [0,1].

Пример: релевантности [3, 2, 0, 1].

```
DCG = (2^3-1)/log2(2) + (2^2-1)/log2(3) + 0 + (2^1-1)/log2(5)
    = 7/1 + 3/1.585 + 1/2.322
    ≈ 7 + 1.893 + 0.431 = 9.324
```

Идеальный порядок [3,2,1,0] даст IDCG; nDCG = DCG/IDCG.

8.13. MAP

Average Precision (AP) — площадь под precision-recall кривой для одного запроса. MAP — среднее AP по запросам. Используется в retrieval, когда важна не только первая находка, но и весь список.

8.14. Triple F1

Для графа знаний важно не только найти узлы, но и правильно соединить их.

Предсказанная тройка `(h, r, t)` считается правильной, если после entity linking и каноникализации она совпала с gold.

```
Triple P = correct triples / predicted triples
Triple R = correct triples / gold triples
Triple F1 = 2PR / (P + R)
```

Это самая честная метрика «граф построен правильно», но она требует размеченных отношений. В нашем бенчмарке её нет, потому что пайплайн не предсказывает типы отношений SciERC.

8.15. Coverage, orphan rate, unresolved link rate

Для vault полезны инженерные метрики:

- **node coverage** — доля gold-концептов, попавших в граф;
- **edge coverage** — доля gold-связей;
- **orphan rate** — доля заметок без ссылок;
- **unresolved link rate** — доля `[[ссылок]]`, не находящих цель;
- **schema conformance** — доля заметок с правильным frontmatter;
- **graph density** — `2E / (N(N-1))` для неориентированного графа;
- **connected components** — число компонент связности.

Эти метрики не заменяют P/R/F1, но показывают, можно ли пользоваться базой.

8.16. Как читать таблицу LitGraphAgent

1. **Concepts exact** — нижняя граница; каноникализация мешает строгому совпадению.
2. **Concepts relaxed** — более честный сигнал смыслового совпадения.
3. **Content relaxed** — то же без Generic-шума; часто самый показательный столбец.
4. **Paper→concept edges** — бипартитная проекция тех же совпадений; не добавляет новой информации о качестве, но показывает, что граф строится.
5. **SOTA SciERC** — другая задача; не ставить в один ряд.

8.17. Частые ошибки

- Сравнивать exact F1 с relaxed F1 и делать вывод «модель стала хуже».
- Сравнивать concept F1 с SciERC entity F1.
- Считать relation F1, если типы отношений не предсказываются.
- Показывать лучший seed и молчать про остальные.
- Забывать про micro/macro.
- Оптимизировать метрику, а не качество графа: например, вернуть 100 концептов, чтобы поднять recall, и разрушить precision.
- Считать 20 абстрактов «лидербордом».

8.18. Главное

Метрика — это линза, а не истина. Одно число не описывает систему. Поэтому в проекте есть несколько сигналов: exact, relaxed, content-only, число концептов, failures, время, QA, тесты. Только вместе они дают честную картину.

9. Запуск, код и команда

9.1. Установка

```
git clone https://github.com/Icold21/LitGraphAgent.git
cd LitGraphAgent
python -m venv .venv

# Windows
.venv\Scripts\activate
# macOS / Linux
source .venv/bin/activate

pip install -e ".[dev]"
```

pypdf входит в основные зависимости: он нужен для `--deep`.

9.2. Настройка `.env`

Создайте `.env` в корне. Пример для локальной Ollama:

```
LLM_API_KEY=ollama
LLM_BASE_URL=http://localhost:11434/v1
LLM_MODEL=qwen3.5:4b

DEFAULT_VAULTS_DIR=./vaults
MAX_PAPERS_PER_RUN=5
MIN_PAPERS_PER_RUN=2
MAX_SEARCH_CANDIDATES=15
CANDIDATE_EVAL_LIMIT=8
CITATION_EXPANSION_LIMIT=2
ENABLE_ELBOW_CUTOFF=true

SEARCH_ROUTER_ENABLED=true
SEARCH_ROUTER_MAX_PRIMARY=2
SEARCH_FALLBACK_MIN_RESULTS=5

MIN_CONCEPT_PAPERS=2
MAX_CONCEPTS_PER_PAPER=8
CONCEPT_BATCH_SIZE=12
BIBLIOGRAPHY_BATCH_SIZE=20

CACHE_ENABLED=true
CACHE_TTL_HOURS=168
```

Для облачной модели:

```
LLM_API_KEY=sk-...
LLM_BASE_URL=https://api.deepseek.com
LLM_MODEL=deepseek-chat
```

Не коммитьте `.env` с реальным ключом. В репозитории есть `.env.example`.

9.3. Проверка окружения

```
litgraph --version
litgraph --team
litgraph --tools
```

`--team` показывает пять ролей и владельцев. `--tools` показывает каталог

поисковых движков, из которого выбирает модель.

9.4. Основные команды

Создать базу

```
litgraph \
  --id "transformer_circuits" \
  --topic "Mechanistic Interpretability and Induction Heads in Transformers" \
  --requirements "Peer-reviewed or high-impact preprints published after 2021" \
  --format "APA 7th" \
  --max-papers 5
```

Обновить базу

```
litgraph \
  --id "transformer_circuits" \
  --update "Sparse Autoencoders and Superposition in Language Models" \
  --max-papers 3
```

Deep-режим

```
litgraph --id "transformer_circuits" --update "Induction heads" --deep
```

Список, проверка, удаление

```
litgraph --list
litgraph --check --id "transformer_circuits"
litgraph --delete --id "transformer_circuits"
```

Переопределить движки

```
litgraph --id "transformer_circuits" --update "Induction heads" --engines arxiv,web
litgraph --id "transformer_circuits" --update "Induction heads" --no-router
```

Отключить кэш или elbow

```
litgraph --id "transformer_circuits" --update "..." --no-cache
litgraph --id "transformer_circuits" --update "..." --no-elbow
```

Логи

```
litgraph --verbose ...
```

9.5. Тесты и линтер

```
pytest -q
ruff check lit_graph tests benchmarks
```

Ожидаемо: 55 passed, ruff без ошибок.

9.6. Бенчмарк

```
# 1. Скачать SciERC
curl -L -o sciERC_raw.tar.gz \
  https://nlp.cs.washington.edu/sciIE/data/sciERC_raw.tar.gz
tar -xzf sciERC_raw.tar.gz

# 2. Запустить
python benchmarks/scierc_benchmark.py \
  --data-dir ./raw_data \
  --limit 20 \
  --seed 42 \
  --model qwen3.5:4b \
  --base-url http://localhost:11434/v1 \
  --output benchmarks/results/scierc_qwen3.5-4b-prompt8.json \
  --checkpoint benchmarks/results/scierc_qwen3.5-4b-prompt8.jsonl

# 3. Построить график
python benchmarks/plot_results.py \
  benchmarks/results/scierc_qwen3.5-4b-prompt8.json \
  --output docs/kp2/assets/metrics_qwen3.5-4b-prompt8.png
```

Бенчмарк умеет resume: если прервать, повторный запуск продолжит с checkpoint. Failures записываются в JSON, а не роняют весь прогон.

9.7. Типичные проблемы

`pull model manifest: file does not exist`

Модели нет в реестре Ollama. Например, `qwen3.5:3b` не существует; есть `qwen3.5:4b`. Проверьте `ollama list`.

Модель очень медленная

Локальная 4B на слабом GPU может давать 1–2 токена/с. Это нормально для ноутбука. Увеличьте `--timeout`, закройте другие GPU-процессы, уменьшите `--limit` для теста.

Таймауты

Поднимите `--timeout` и `--retries`. Для больших баз используйте checkpoint.

JSON-ошибки

Санитайзер восстанавливает большинство случаев, но если модель стабильно ломает JSON, попробуйте другую модель или уменьшите батч.

Нет PDF в deep-режиме

Не все статьи open-access. Deep-режим деградирует: статья обрабатывается по абстракту, grounding пропускается.

Битые ссылки в Obsidian

Запустите `litgraph --check --id ...`. QA покажет unresolved links, orphans и frontmatter issues.

Кириллица в PowerShell

Если вывод Python ломается, задайте:

```
$env:PYTHONIOENCODING='utf-8'
```

9.8. Карта кода

Файл	Зачем
`lit_graph/agent/core.py`	Lead, жизненный цикл, CLI-интеграция
`lit_graph/agent/workflow.py`	LangGraph, узлы, рёбра, activity
`lit_graph/agent/graph_state.py`	`ResearchState`, events, reducer
`lit_graph/agent/roles.py`	Роли и владельцы
`lit_graph/agent/retrieval_agent.py`	Поиск, citations, PDF
`lit_graph/search/`	Клиенты, каталог, router, ranking, PDF
`lit_graph/agent/prompt_agent.py`	LLM-операции
`lit_graph/agent/validator.py`	Валидация кандидатов
`lit_graph/agent/synthesizer.py`	Синтез, определения, библиография, обзор
`lit_graph/llm/prompts.py`	Все промпты
`lit_graph/llm/providers.py`	Провайдеры LLM
`lit_graph/agent/graph_agent.py`	Онтология, rebuild, flush
`lit_graph/agent/concepts.py`	Каноникализация
`lit_graph/storage/obsidian_vault.py`	Запись заметок, state
`lit_graph/storage/base_manager.py`	Жизненный цикл баз
`lit_graph/agent/evaluation_agent.py`	QA-агент
`lit_graph/qa/vault_checks.py`	Проверки
`lit_graph/models.py`	Модели данных
`lit_graph/cli.py`	CLI
`lit_graph/config.py`	Настройки
`tests/`	55 тестов
`benchmarks/`	SciERC harness и результаты

9.9. Как добавить нового агента

1. Добавить роль в `agent/roles.py`.

2. Написать сервис-класс с обычными методами.
3. Инстанцировать в `agent/core.py`.
4. Добавить узлы и рёбра в `agent/workflow.py`.
5. Добавить тест.
6. Обновить `CONTRIBUTORS.md`, `docs/AGENT_TEAM.md`, `README.md`, `.github/CODEOWNERS`.

Инварианты: агент не пишет данные другого агента напрямую; все промпты — в `llm/prompts.py`; все записи vault — через `GraphAgent`; QA не мутирует состояние.

9.10. Как добавить поисковый движок

1. Описать движок в `search/tools.py`.
2. Добавить клиент в `search/`.
3. Подключить в `search/engine.py`.
4. Добавить в каталог и тесты.
5. Проверить `litgraph --tools`.

Не добавляйте ветвление «если движок X» внутрь агента: каталог должен оставаться данными.

9.11. Как изменить промпт

1. Найти функцию в `llm/prompts.py`.
2. Изменить текст, сохранив требование JSON.
3. Проверить, что `synthesizer.py`/`validator.py` действительно используют эту функцию (раньше был дубликат inline-промпта).
4. Прогнать `pytest` и бенчмарк на 5–10 абстрактах.
5. Обновить метрики и документацию.

9.12. Команда и роли

Участник	Роль	Зона
Иван Холодило	Lead AI Engineer	Архитектура, LangGraph, CLI, состояние
Вадим Лялькин	Graph & Knowledge Engineer	Онтология, `[[wikilinks]]`, Obsidian
Нодар Чигладзе	Data / IR Engineer	arXiv, Semantic Scholar, web, ranking, PDF
Давид Чигладзе	QA / Evaluation Engineer	QA, тесты, CI, бенчмарк
Илья Синко	LLM Engineer	Промпты, валидация, синтез

Подробнее: `CONTRIBUTORS.md`, `docs/kp2/CONTRIBUTIONS.md`, `.github/CODEOWNERS`.

9.13. Дорожная карта

Дата	Этап	Статус
6 октября	MVP: поиск, фильтрация, заметки	Выполнено

5 ноября	Модульный поиск и первые метрики	Выполнено
25 ноября	Тесты, стабилизация, калибровка	Частично
15 декабря	Финальный релиз, демо, документация	В работе

До финала:

- 100 абстрактов, 3 seed, 2–3 модели;
- triple-level F1, если появятся типы отношений;
- precision@k/nDCG для поиска;
- coverage, orphan rate, unresolved link rate;
- p95 latency и стоимость прогона;
- демо и скриншоты.

9.14. Риски

Риск	Причина	План
Узкая оценка	20 абстрактов, один seed	Расширить выборку
Нет relation F1	Пайплайн не предсказывает типы	Либо добавить типы, либо честно не считать
Локальная модель медленная	4B на слабом GPU	Таймауты, батчинг, меньший `--limit`
Внешние API недоступны	Сеть/лимиты	Retry, cache, fallback
Каноникализация ошибается	Эвристики	QA, ручная проверка, тесты
Вклад участников	История git	Коммиты по зонам, подтверждение

9.15. Что считать успехом

Не «модель выдала красивый граф», а:

- база воспроизводится;
- QA не находит критичных ошибок;
- метрики стабильны на нескольких seed;
- пользователь может открыть vault и найти нужное;
- каждый агент можно объяснить и протестировать отдельно.

10. Глоссарий, FAQ и приложение

10.1. Глоссарий

LLM — большая языковая модель; вероятностный генератор текста.

Промпт — инструкция модели: роль, данные, правила, формат ответа.

Токен — единица текста для модели.

Контекст — весь текст, который модель видит в одном запросе.

Температура — параметр случайности генерации.

JSON — текстовый формат структурированных данных.

Ollama — локальный сервер моделей с OpenAI-совместимым API.

Провайдер LLM — сервис/сервер, к которому обращается проект.

Агент — компонент с ограниченной ролью.

Оркестрация — управление порядком вызовов агентов.

LangGraph — библиотека для графа workflow.

State — данные, передаваемые между узлами.

Reducer — функция объединения старого и нового значения поля.

Checkpoint — сохранённая точка состояния.

Knowledge graph — граф знаний: узлы-сущности и рёбра-отношения.

Узел — вершина графа.

Ребро — связь между узлами.

Триплет — тройка «голова, отношение, хвост».

Онтология — допустимые типы узлов и рёбер.

Каноникализация — приведение разных написаний к одному виду.

Vault — папка Obsidian с Markdown-заметками.

Markdown — простой текстовый формат разметки.

Frontmatter — YAML-блок метаданных в начале заметки.

Wikilink — ссылка `[[Заметка]]` внутри vault.

Stub — лёгкая заметка-заглушка для внешней ссылки.

Coverage — доля gold-сущностей/связей, попавших в граф.

Orphan — узел без ссылок.

Unresolved link — ссылка, не находящая 目标.

Retrieval — поиск кандидатов.

Дедупликация — удаление одинаковых статей.

Ранжирование — сортировка по полезности.

Elbow cutoff — отсечение 低-impact хвоста по «локтю».

Fallback — запасной движок/вариант.

Sub-query — уточняющий подзапрос.

Precision — доля правильного среди найденного.

Recall — доля найденного среди правильного.

F1 — гармоническое среднее precision и recall.

Exact — точное совпадение после нормализации.

Relaxed — совпадение по Jaccard ≥ 0.5.

Micro — агрегирование TP/FP/FN по всем примерам.

Macro — среднее метрик по примерам.

MRR — среднее обратных рангов.

Hits@k — доля запросов с правильным ответом в top-k.

nDCG — метрика ранжирования с градацией релевантности.

MAP — среднее Average Precision.

Triple F1 — F1 по тройкам графа.

QA — проверка качества.

CI — автоматический запуск тестов и линтера.

pytest — фреймворк тестов.

ruff — линтер Python.

SciERC — бенчмарк scientific knowledge-graph construction.

SOTA — state of the art, лучший известный результат.

10.2. FAQ

Это заменяет ChatGPT?

Нет. LitGraphAgent не чат, а конвейер с фиксированными шагами. LLM здесь — исполнитель операций, а не свободный собеседник.

Почему не просто отправить тему в ChatGPT?

Потому что нужен воспроизводимый граф, ссылки, библиография, состояние и проверки. Чат не даёт структуру, не сохраняет `_state.json` и не проверяет целостность vault.

Почему локальная модель?

Локальная модель не отправляет тексты в облако, дешевле в эксплуатации и

работает без интернета (после загрузки). Минус — качество и скорость.

Почему qwen3.5:4b, а не qwen3.5:3b?

Модели `qwen3.5:3b` нет в реестре Ollama (`pull model manifest: file does not exist). Поэтому используется 4B. 3B-сравнение сделано на qwen2.5:3b`.

Почему exact F1 такой низкий?

Потому что система возвращает канонические концепты, а gold — сырые спаны. `word sense disambiguation (WSD) systems` и `word sense disambiguation` — это одно и то же по смыслу, но не по строке. Relaxed и content-only показывают более честную картину.

Почему не считается relation F1?

Пайплайн не предсказывает типы отношений SciERC. Считать relation F1 было бы фикцией. Это ограничение честно зафиксировано.

Почему 20 абстрактов?

Это regression sample, а не лидерборд. 20 хватает, чтобы ловить регрессии, но не для статистической значимости. До КП3 планируется 100 абстрактов и 3 seed.

Почему prompt v2 медленнее?

Он просит 6–8 концептов вместо 3 примеров, поэтому модель генерирует больше текста. Время выросло с ~31 до ~59 минут на 20 абстрактов. Это trade-off качества и скорости.

Можно ли использовать другую модель?

Да. Любой OpenAI-совместимый endpoint: DeepSeek, OpenAI, Groq, OpenRouter, vLLM. Достаточно поменять `.env`.

Можно ли использовать без Obsidian?

Vault — это папка Markdown. Obsidian нужен только для graph view и удобной навигации. Без него файлы всё равно читаются.

Что будет, если LLM вернёт плохой JSON?

Санитайзер попробует восстановить, затем retry, затем fallback. Если совсем плохо — статья может остаться без части полей, а QA покажет warning.

Что будет, если сеть упала?

Retrieval сделает retry/backoff, затем пропустит кандидата. База может быть меньше, но запуск не упадёт.

Как проверить, что база не сломана?

```
litgraph --check --id "your_base"
```

QA покажет errors/warnings.

Где хранится состояние?

В `vaults/<id>/_state.json`. Запись атомарная.

Можно ли обновлять базу частями?

Да, `--update` для этого и нужен. Старые определения не перезаписываются.

Почему в графе есть внешние заметки?

Это stubs для цитирований, которых нет в базе. Они нужны, чтобы `[[ссылки]]`

не повисали.

Как добавить новый движок?

Описать в `search/tools.py`, добавить клиент, подключить в `search/engine.py`,

добавить тест. Не встраивать условие в агента.

Как добавить нового агента?

Роль, сервис, инстанс в `core.py`, узлы/рёбра в `workflow.py`, тест, docs,

CODEOWNERS.

10.3. Приложение А. Пример `_state.json`

```
{
  "base_id": "transformer_circuits",
  "topic": "Mechanistic Interpretability and Induction Heads",
  "requirements": "Peer-reviewed or high-impact preprints after 2021",
  "citation_format": "APA 7th",
  "overall_summary_en": "The literature converges on...",
  "papers": {
    "2401.00001": {
      "id": "2401.00001",
      "title": "Example Paper",
      "authors": ["A. Author", "B. Author"],
      "year": 2024,
      "venue": "arXiv",
      "doi_or_url": "https://arxiv.org/abs/2401.00001",
      "source_engine": "arxiv",
      "citation_count": 42,
      "influential_citation_count": 7,
      "citation_velocity": 10.5,
      "influential_ratio": 0.186,
      "objective_scientific_score": 13.2,
      "normalized_weight": 0.71,
      "summary_en": "The paper studies...",
      "key_findings_en": ["Finding 1", "Finding 2"],
      "methodology_en": "The authors...",
      "limitations_en": "Small sample...",
      "extracted_concepts": ["Induction Head", "Transformer"],
      "extracted_concepts_all": ["Induction Head", "Transformer", "Attention"],
      "formatted_citation": "Author, A. (2024). Example Paper..."
    }
```

```
  },
  "concepts": {
    "Induction Head": {
      "name": "Induction Head",
      "summary_en": "A circuit that...",
      "aggregated_score": 4.2,
      "normalized_weight": 0.83,
      "mentions": [
        {
          "concept_name": "Induction Head",
          "paper_title": "Example Paper",
          "page_number": 3,
          "context_snippet": "We identify an induction head..."
        }
      ]
    }
  },
  "edges": [
    {
      "source_title": "Example Paper",
      "target_title": "Another Paper",
      "relation_type": "cites"
    }
  ],
  "search_plan": {
    "primary": ["arxiv", "semantic_scholar"],
    "fallback": ["web"],
    "reason": "Fresh preprints plus citation metadata.",
    "source": "llm"
  },
  "created_at": "2026-11-09T10:00:00+00:00",
  "updated_at": "2026-11-09T10:05:00+00:00"
}
```


10.4. Приложение B. Пример заметки источника

```
---
id: "2401.00001"
title: "Example Paper"
authors: ["A. Author"]
year: 2024
venue: "arXiv"
url: "https://arxiv.org/abs/2401.00001"
source_engine: "arxiv"
citation_count: 42
citation_velocity: 10.5
objective_score: 13.2
node_weight: 0.71
type: literature_note
---

# Example Paper

> 📊 **Scientometric Score:** `13.2` | **Continuous Weight:** `0.71`

## 📌 Formatted Citation

> Author, A. (2024). Example Paper...

## 🧠 Executive Summary (English)

The paper studies...

## 🔍 Key Findings & Contributions

- Finding 1
- Finding 2
```

```
## ⚙️ Methodology & Limitations

**Methodology:** The authors...

**Limitations:** Small sample...

## 🌐 Knowledge Graph Connections

**Concepts:** [[Induction Head]], [[Transformer]]

**Related Literature:** None

**References (Outward):** [[Another Paper]]

**Citations (Inward):** None

## 📄 Abstract

> ...
```

10.5. Приложение C. Пример концепт-хаба

```
---
concept: "Induction Head"
aggregated_score: 4.2
node_weight: 0.83
type: concept_hub
---

# Concept: Induction Head

> 🌐 **Aggregated Score:** `4.2` | **Continuous Weight:** `0.83`

## 📖 Concept Definition & Consensus

A circuit that...

## 📍 Grounded Citations & Page References:

- [[Example Paper]] (**p. 3**) — *"We identify an induction head..."*
```

10.6. Приложение D. Пример QA-отчёта

```
{
  "summary": "0 errors, 2 warnings (notes=42, sources=12, concepts=28, links=96)",
  "stats": {
    "notes": 42,
    "sources": 12,
    "concepts": 28,
    "external_refs": 2,
    "links": 96,
    "unresolved_links": 0,
    "orphans": 2
  },
  "issues": [
    {
      "severity": "warning",
      "code": "orphan_note",
      "message": "Note has no inbound or outbound links",
      "path": "Concepts/Example.md"
    }
  ]
}
```

10.7. Приложение E. Промпт v2

```
Synthesize the following paper strictly in English:
Title: ...
Authors: ...
Year: ...
Venue: ...
Abstract: ...

Concept extraction rules:
- Return 6-8 key scientific concepts that are explicitly mentioned in the abstract.
- Prefer specific methods, tasks, metrics, materials, datasets and technical terms.
- Prefer canonical multi-word names over bare acronyms.
- Avoid generic words such as system, approach, method, algorithm or result.
- Return each concept once.

Return strict JSON with concepts first:
{
  "extracted_concepts": ["Specific concept 1", "Specific concept 2", "Specific concept 3"],
  "summary_en": "Deep single-paragraph executive summary in English",
  "key_findings_en": ["Key finding 1", "Key finding 2", "Key finding 3"],
  "methodology_en": "Methodology summary in English",
  "limitations_en": "Limitations identified in English"
}
```

10.8. Приложение F. Шпаргалка команд

```
# окружение
litgraph --version
litgraph --team
litgraph --tools

# создать
litgraph --id "base" --topic "..." --requirements "..." --format "APA 7th" --max-papers 5

# обновить
litgraph --id "base" --update "..." --max-papers 3

# deep
litgraph --id "base" --update "..." --deep

# проверить
litgraph --check --id "base"

# список/удаление
litgraph --list
litgraph --delete --id "base"

# тесты
pytest -q
ruff check lit_graph tests benchmarks

# бенчмарк
python benchmarks/scierc_benchmark.py --data-dir ./raw_data --limit 20 --seed 42 --model
qwen3.5:4b --output benchmarks/results/run.json
python benchmarks/plot_results.py benchmarks/results/run.json
```

10.9. Что читать дальше

- `README.md` — быстрый старт и верхнеуровневые метрики.
- `docs/ARCHITECTURE.md` — архитектура и диаграммы.
- `docs/RETRIEVAL.md` — поиск и ранжирование.

- `docs/LLM_LAYER.md` — промпты и синтез.
- `docs/GRAPH_MODEL.md` — онтология и каноникализация.
- `docs/KG_METRICS.md` — метрики KG-бенчмарков и SOTA-ориентиры.
- `docs/kp2/KP2_REPORT.md` — отчёт КП2.
- `benchmarks/README.md` — протокол SciERC.

10.10. Заключение

Если вы дошли до конца, у вас есть полная карта проекта: зачем он нужен, из каких агентов состоит, как проходит статья от поиска до заметки, что именно измеряет бенчмарк и где лежит код. Этого достаточно, чтобы включиться в работу, объяснить систему на защите и не потеряться в первый день.

Главное, что стоит запомнить:

- LitGraphAgent — это конвейер, а не «магический ИИ». Каждый шаг можно проверить.
- LLM — вероятностный компонент; поэтому есть валидация, JSON-санитайзер, fallback-и и QA.
- Граф знаний держится на каноникализации, ссылках и дедупликации.
- `_state.json` — источник правды между запусками.
- Бенчмарк измеряет только то, что система реально предсказывает; relation F1 без типов отношений был бы фикцией.
- 20 абстрактов — regression sample, а не лидерборд.

Если понадобится продолжить проект, начните с `README.md`, затем откройте `docs/ARCHITECTURE.md` и `docs/kp2/KP2_REPORT.md`, а после — код агентов в том порядке, в котором они вызываются: Lead → Retrieval → Prompt → Graph → QA.

Контакты и ссылки:

- Репозиторий: <https://github.com/Icold21/LitGraphAgent>
- Команда: AI-RIldium
- Лицензия: MIT
- Дата сборки гайда: ноябрь 2026

Спасибо, что дочитали. Дальше — только код, эксперименты и вопросы.